

Understanding Unified Modelling Language (UML)

OMG

- Object Management Group, founded in 1989
- Consortium of 800 industries (eg. IBM, HP, Apple, etc.) and many universities.
- Produces specifications of reference architectures, eg. CORBA
- Other specifications: UML, various MDA technologies
- UML as managed by OMG is a standard de facto in continuous evolution.
- UML1.4 (onwards) is a ISO standard.

Main UML specification documents

- **Superstructure**: defines the UML elements (diagrams, etc.)
- **Infrastructure**: defines the UML core metamodel
- **OCL** (Object Constraint Language):
formal language for writing predicates, constraints, and formulas inside diagrams
- **XMI** (XML Metadata Interchange):
Document Type Definition (DTD) for UML models
- UML Diagram Interchange: XMI + graphic info

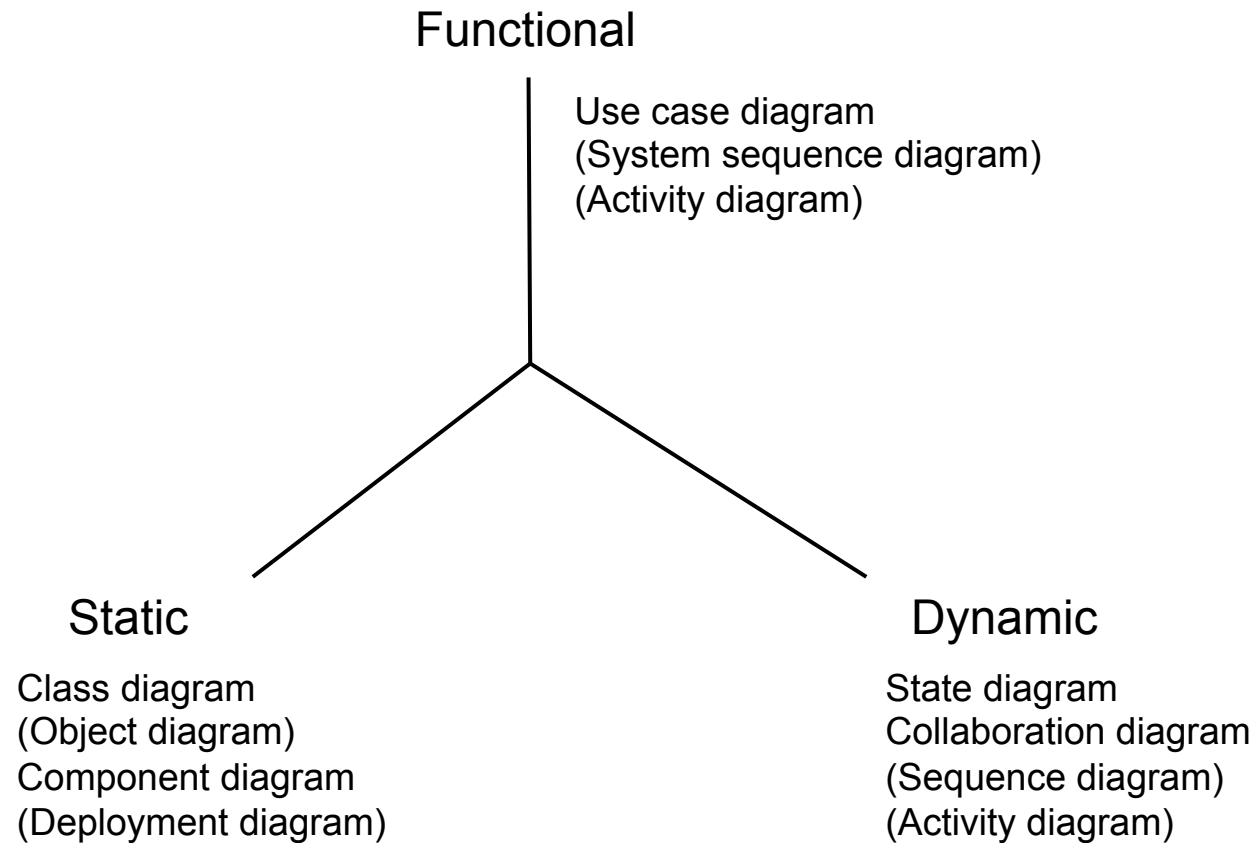
Structure and Behavior

- UML focesses on two aspects of object oriented models:
 - Structure and Behavior
- It aims at visualizing both.

Describing the structure of software

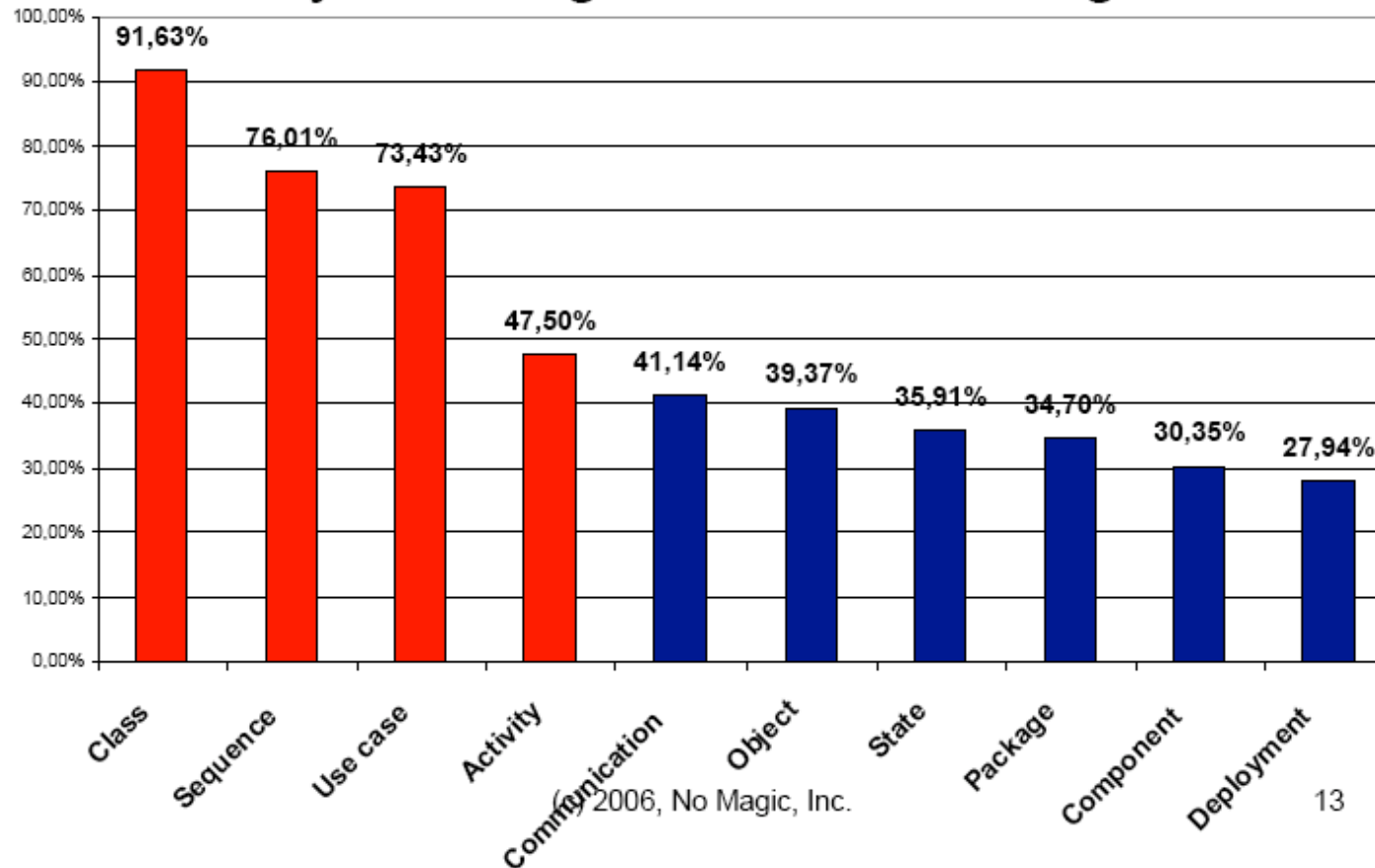
- The description of structure offers an account of what a system is made of, in terms of both its parts and the relationships among them.
- A structure may be a hierarchy featuring one-to-many relationships, or
- A network featuring many-to-many links.
- In short, artifacts and the way they are connected to each other to define a system

Three modeling axes



Usage survey

Survey on Usage of UML 1.x Diagrams



Main diagrams

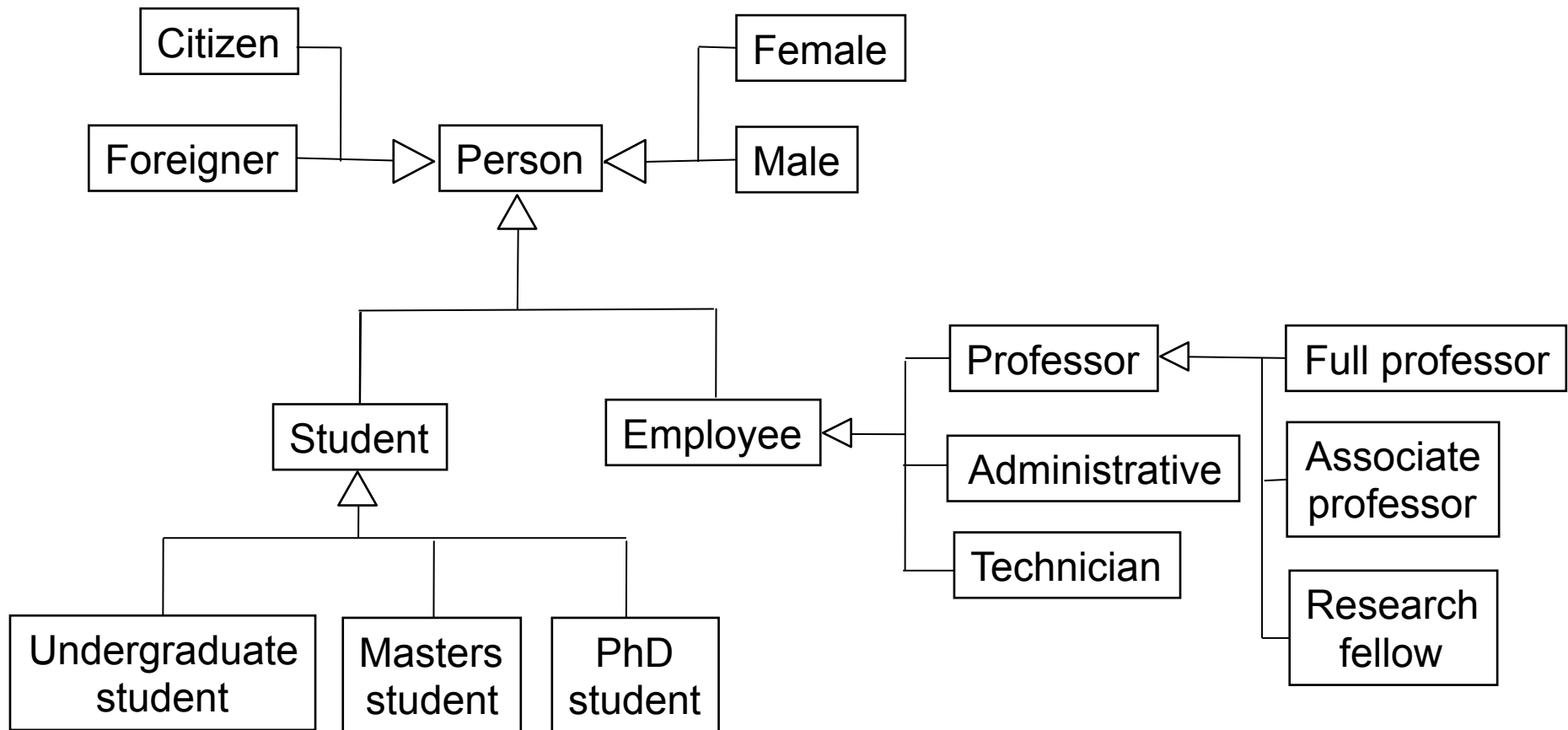
The main diagrams that are used in most views are :

- **Use case** diagram
- **Class** diagram
- **Sequence** diagram
- **Activity** diagram

Example

- A university is an organization where some persons work, some other study
- There are several types of roles and grouping entities
- We say nothing about behaviors, for the moment

A taxonomy

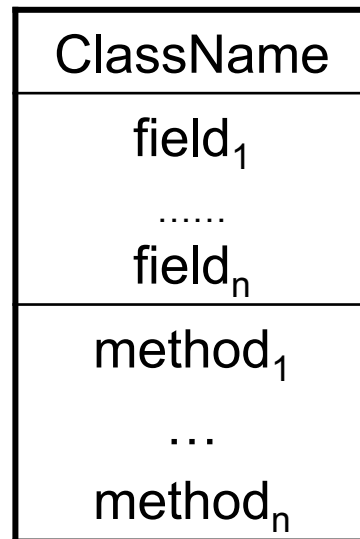


Class diagram

- Most common diagram in OO modeling
- A class diagram is a graph including:
 - Nodes representing **classes (types of objects)**
 - Nodes can have just a name, or expose some internal structure
 - Links representing **relationships among classes**
 - Inheritance
 - Association
 - Aggregation or composition
 - Dependency
 - Links can have multiplicities and/or names for roles played by participants

Notation for classes

- The notation for classes is a rectangular box with three compartments



The top compartment shows the class **name**

The middle compartment contains the declarations of the fields, or *attributes*, of the class

The bottom compartment contains the declarations of the *methods* of the class

Example

A point defined by classes at three different abstraction levels

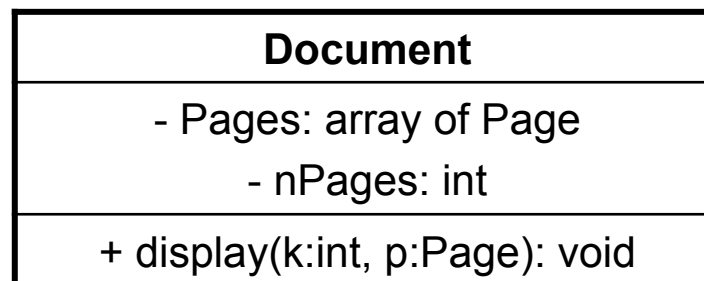
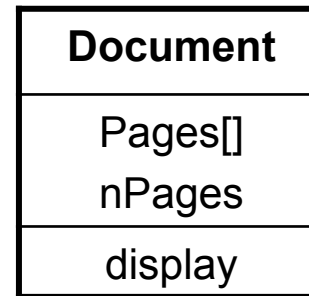
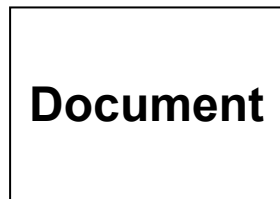
Point

Point
x
y
Move

Point
- x: int - y: int
+ move(dx: int, dy: int): void

Example

A document defined by classes at three different abstraction levels



UML Class Diagrams

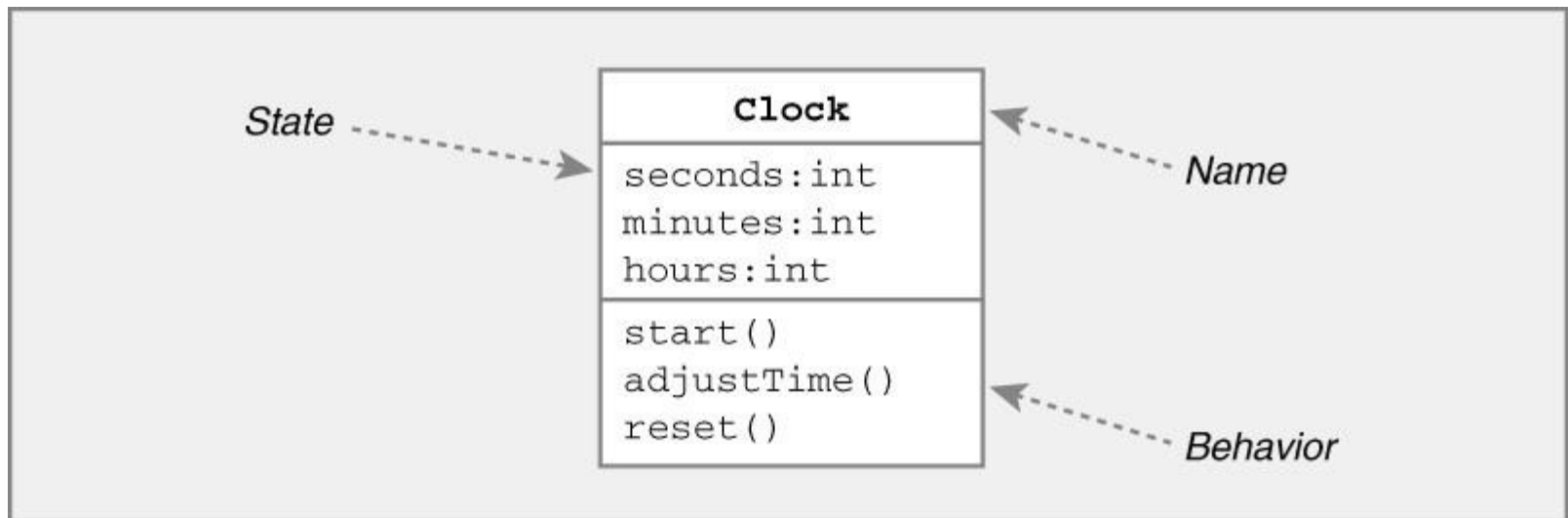
■ Represent the (static) structure of the system

■ General In Java

■ Name Name

■ State Variables

■ Behavior Methods



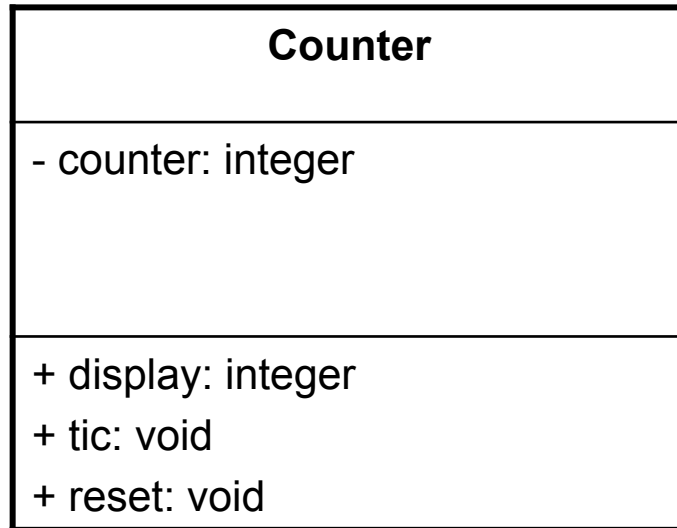
Exercise



Draw a class diagram for the following Java code

```
class Person {  
    private String name;  
    private Date birthday;  
    public String getName() {  
        // ...  
    }  
    public Date getBirthday() {  
        // ...  
    }  
}
```

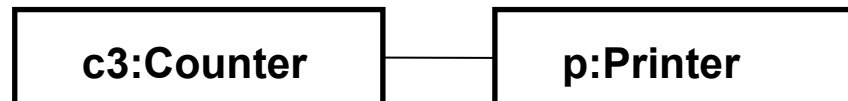

A counter class



A class in UML

```
class Counter{  
    private counter: integer;  
    public integer display()  
        {return counter};  
    public void tic()  
        {counter = counter + 1};  
    public void reset()  
        {counter = 0};  
}
```

A corresponding class
in a programming language



Using an object of type class
in an object oriented system

Relationships Between Classes

■ **Association**  OR 

- Permanent, structural, “has a”
- Solid line (arrowhead optional)

■ **Dependency** 

- Temporary, “uses a”
- Dotted line with arrowhead

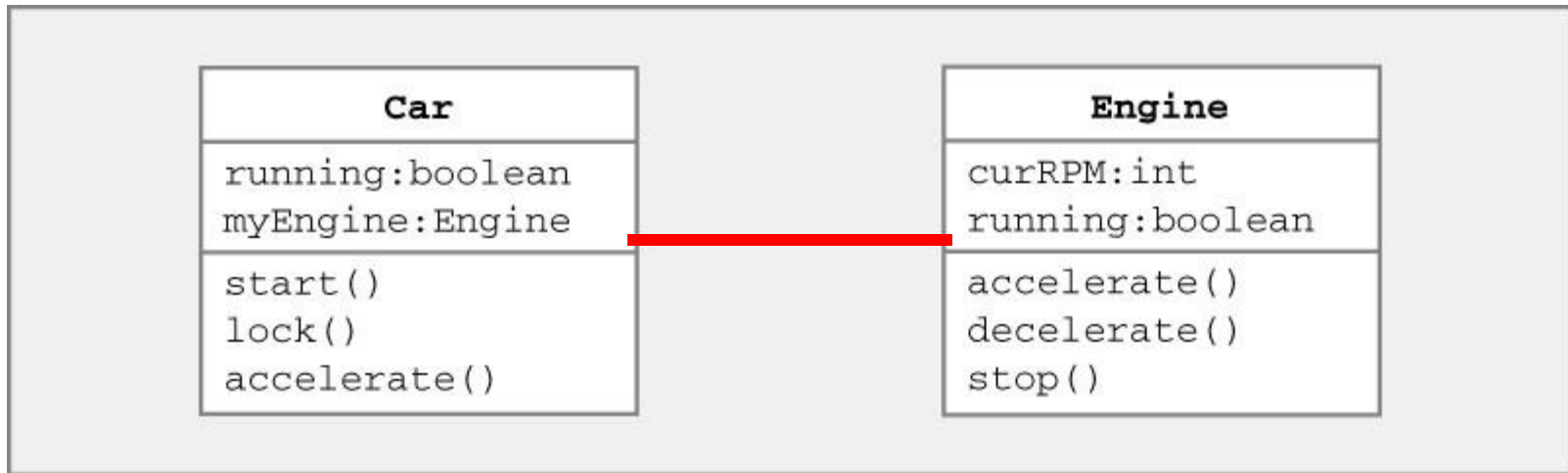
■ **Generalization** 

- Inheritance, “is a”
- Solid line with open (triangular) arrowhead

■ **Composition and Aggregation**

Association

- Denotes permanent, structural relationship
- State of class A contains class B
- Represented by solid line (arrowhead optional)



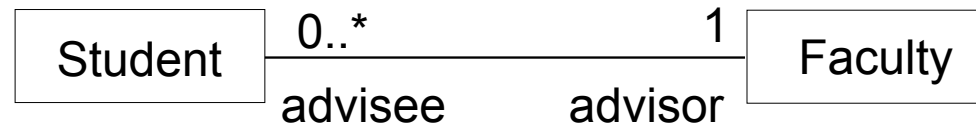
Car and Engine classes know about each other

Association and roles

- The simplest relationship among classes is the *association*
- An association represents a structural relationship that connects two classes
- It means that the two classes “know” each other: all objects in a class have some relationship with some object(s) in the other class
- An association can have a name, which usually describes its role

Roles and multiplicity

- An association line may have a *role name* and a *multiplicity specification*
- The multiplicity specifies an integer interval, e.g.,
 - $l..u$ closed (inclusive) range of integers
 - i singleton range
 - $0..*$ nonnegative integer, i.e., 0, 1, 2, ...



Multiplicity of Associations

- Some relationships may be quantified
- Multiplicity denotes how many objects the source object can legitimately reference
- Notation
 - * $\Rightarrow$ 0, 1, or more
 - 5 $\Rightarrow$ 5 exactly
 - 5..8 $\Rightarrow$ between 5 and 8, inclusive
 - 5..* $\Rightarrow$ 5 or more

Multiplicity of Associations

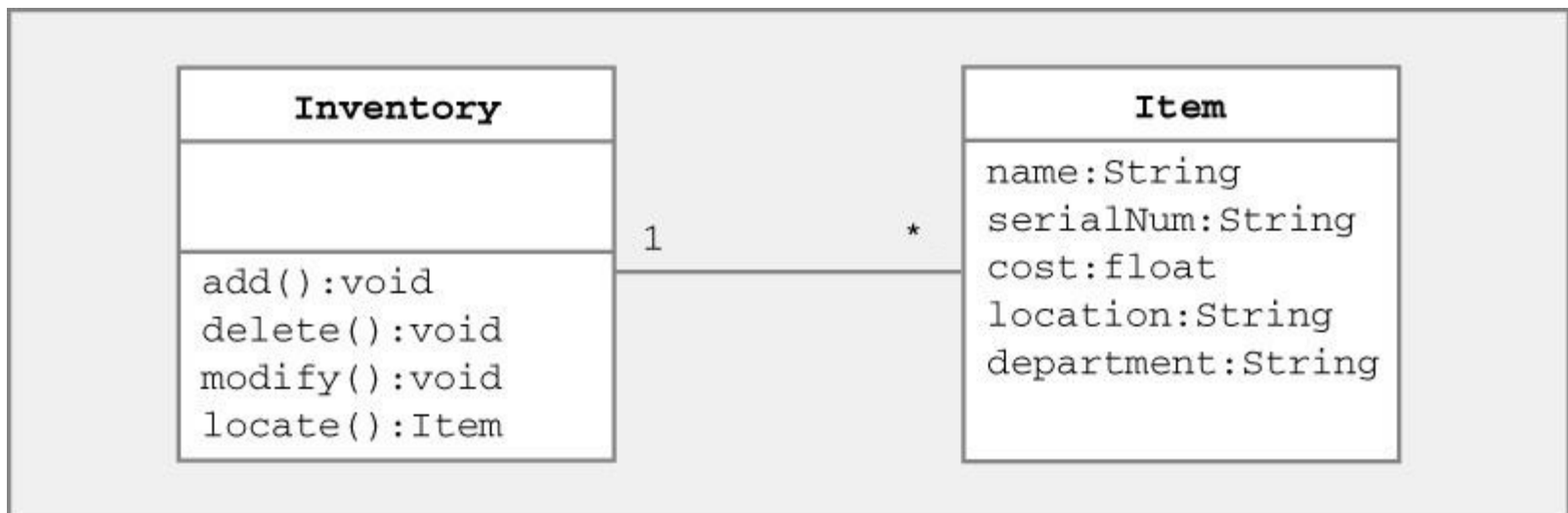
■ Many-to-one

- Bank has many ATMs, ATM knows only 1 bank



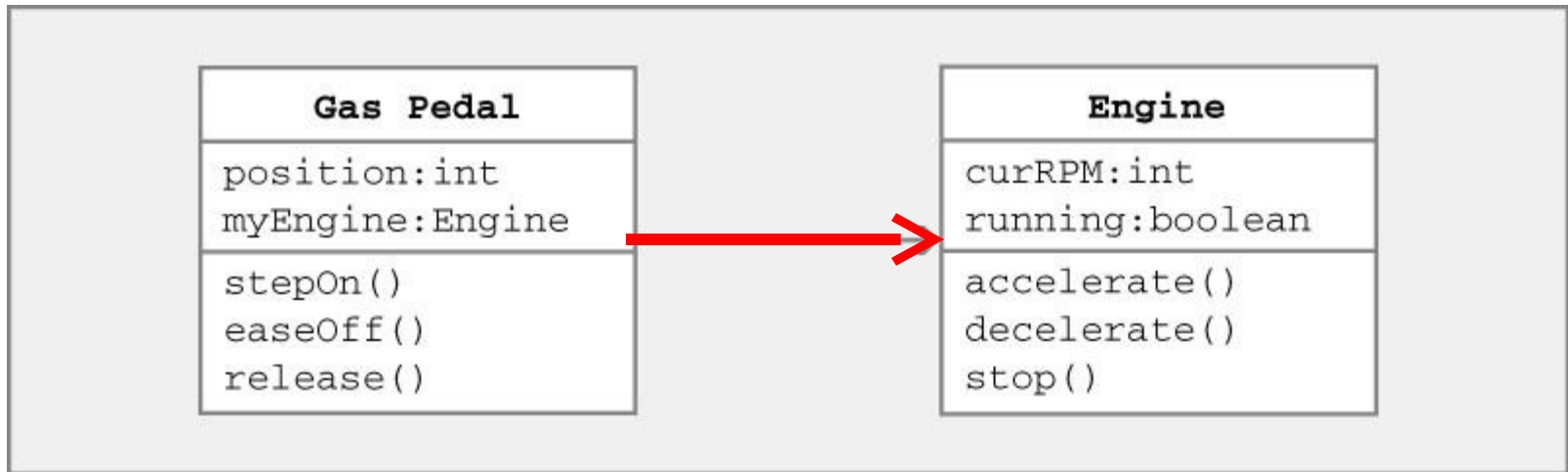
■ One-to-many

- Inventory has many items, items know 1 inventory



Associations w/ Navigation Information

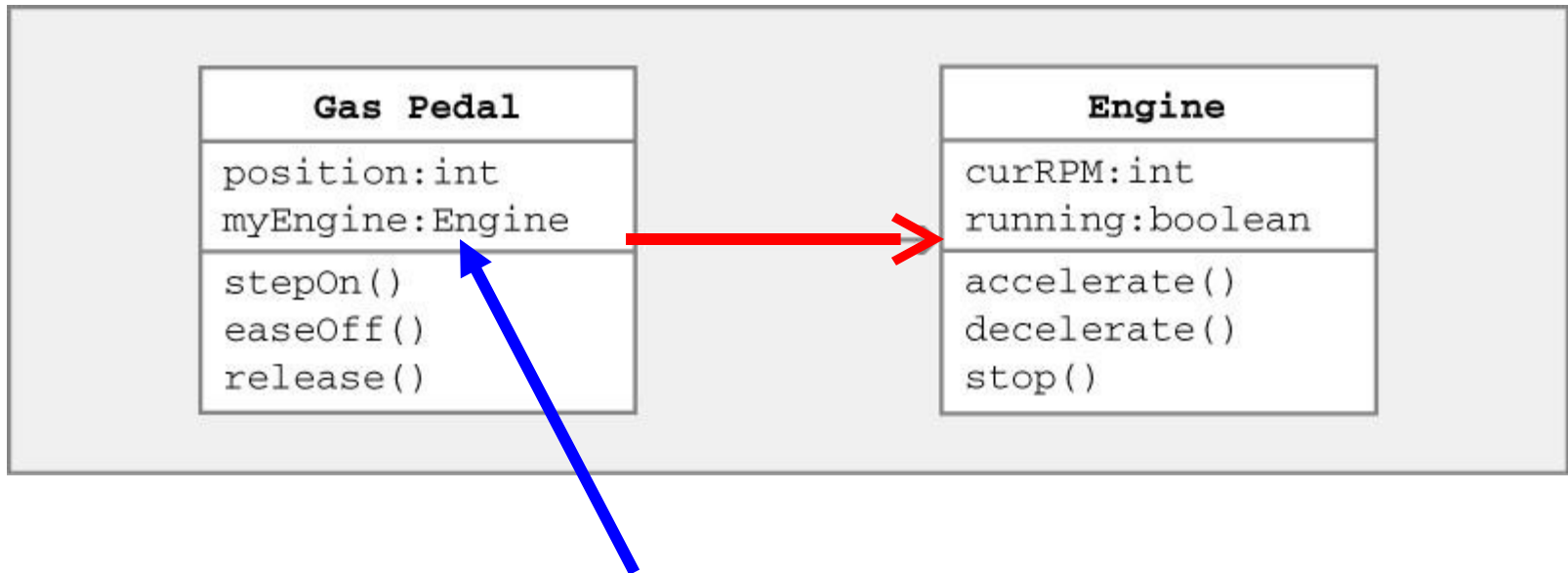
- Can indicate direction of relationship
- Represented by solid line with arrowhead



Gas Pedal class knows about Engine class
Engine class doesn't know about Gas Pedal class

Associations w/ Navigation Information

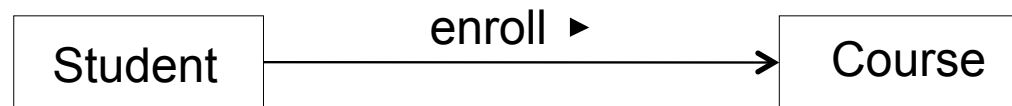
- Denotes “**has-a**” relationship between classes
- “Gas Pedal” **has a** “Engine”



State of Gas Pedal class contains instance of Engine class ⇒ can invoke its methods

Association with direction

- An association may have an optional label consisting of a name and a direction arrow
- The direction arrow indicates the direction of association with respect to the name

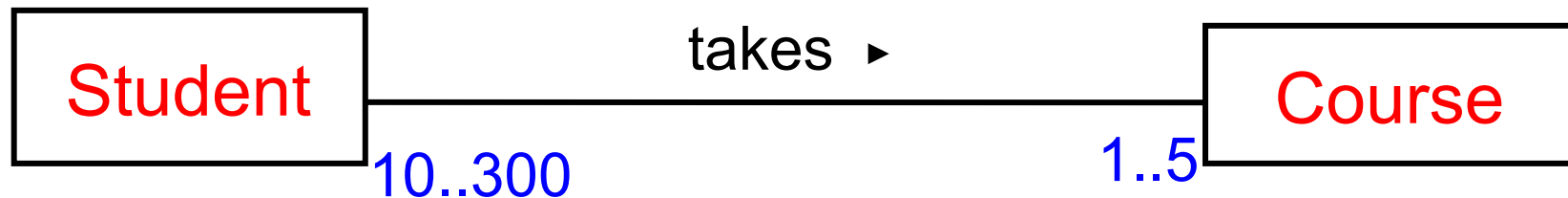


An arrow may be attached to the end of path to indicate that *navigation* is supported in that direction:

Student knows Course, Course does not know Student

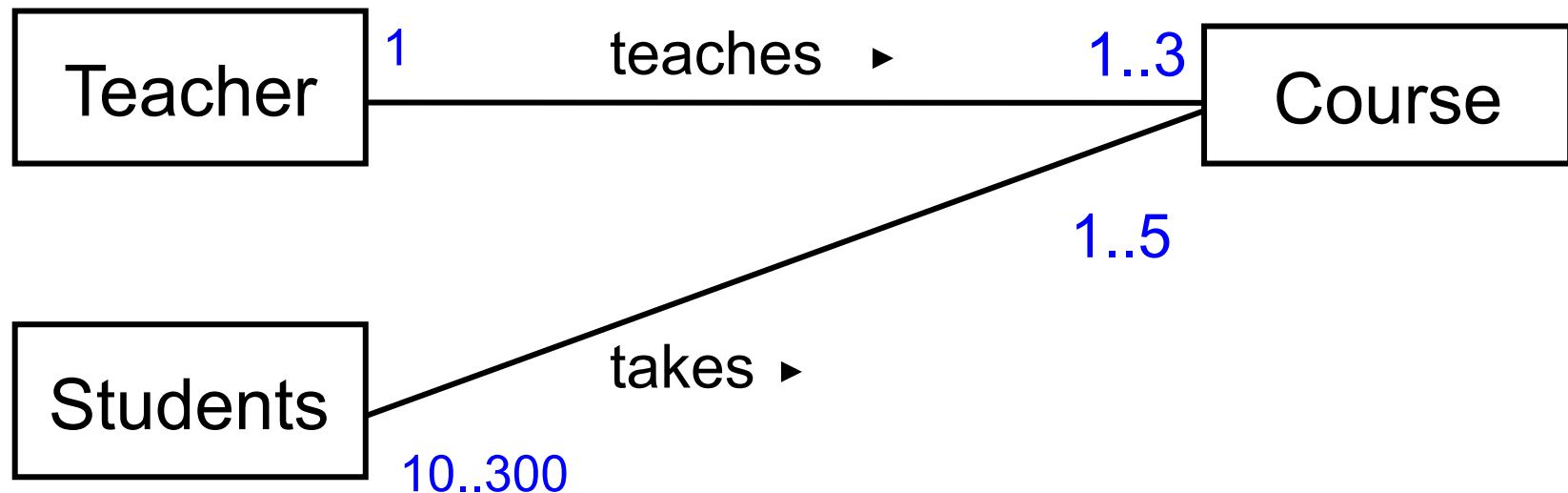
Association example

- A **Student** can take up to **five Courses**
- Every Student has to be enrolled in at least **one** course
- Up to **300** students can enroll in a course
- A class should have **at least 10** students



Example

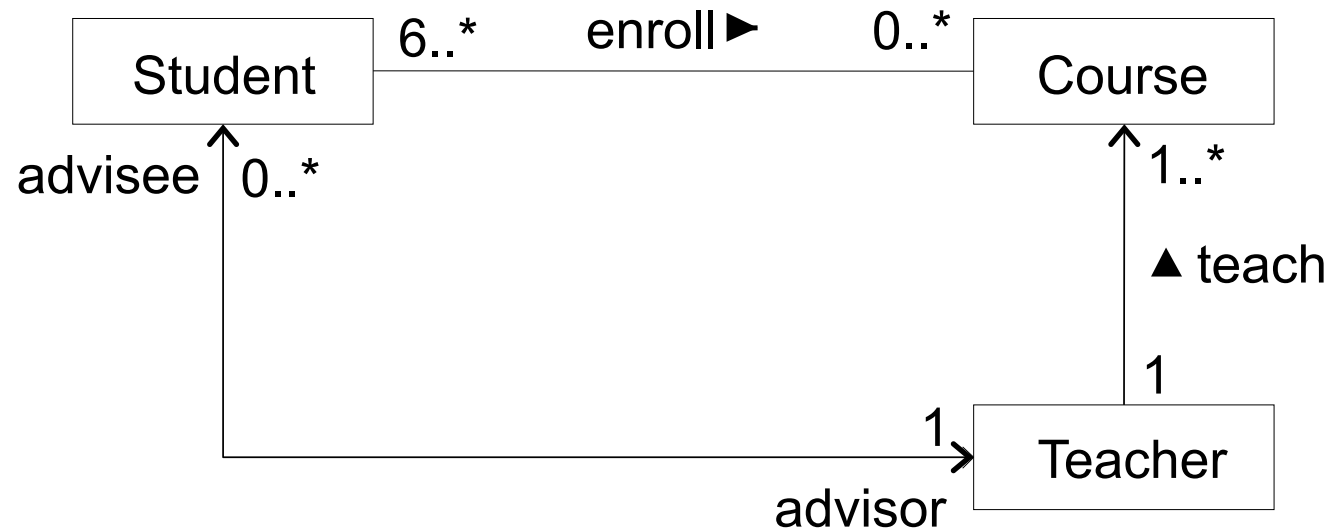
- A teacher teaches 1 to 3 courses
- Each course is taught by only one teacher
- A student can take between 1 to 5 courses
- A course can have 10 to 300 students



Exercise



Explain the meaning of this diagram



Dependency

- Denotes **dependence** between classes
- Always directed (Class **A** depends on **B**)
- Represented by dotted line with arrowhead



A depends on B

Dependency

- Caused by class methods
- Method in Class **A** temporarily “**uses a**” object of type Class **B**
- Change in Class **B** may affect class **A**



A uses object of class B

Dependency

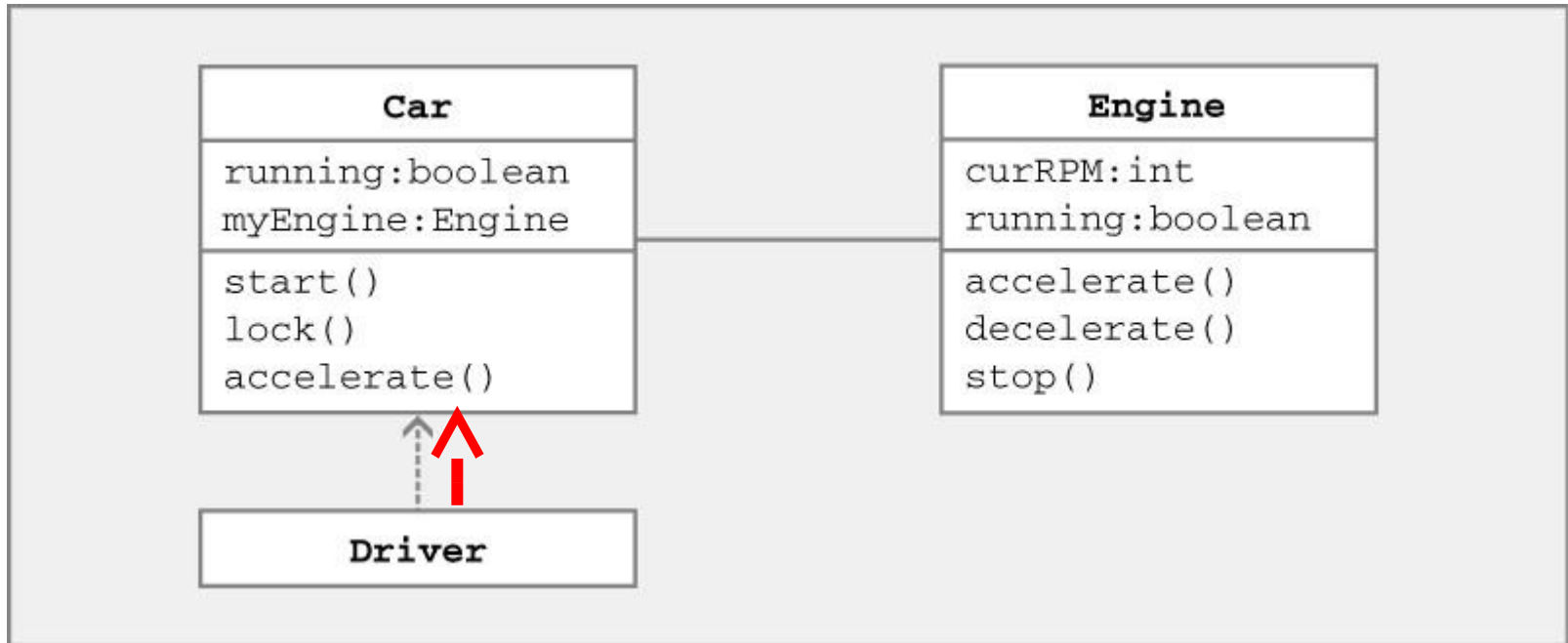
- Dependence may be caused by
 - Local variable
 - Parameter
 - Return value
- Example

```
Class A {  
    B Foo(B x) {  
        B y = new B();  
        return y;  
    }  
}
```



```
Class B {  
    ...  
    ...  
    ...  
}
```

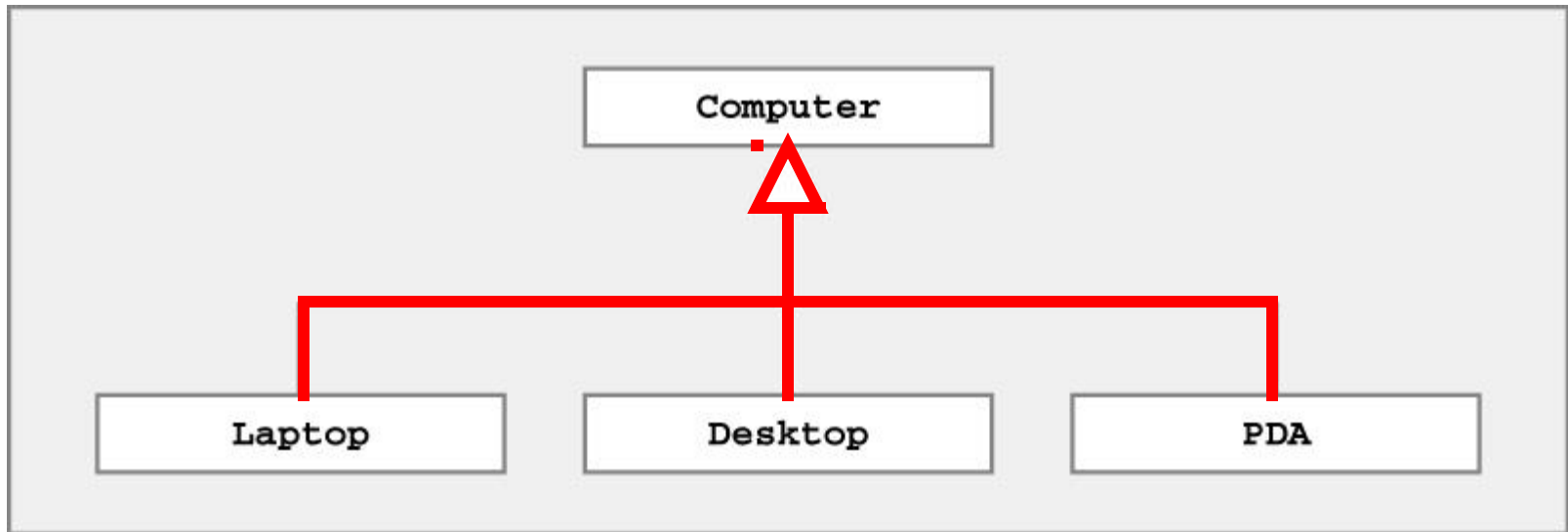

Dependency Example



Class Driver depends on Class Car

Generalization

- Denotes **inheritance** between classes
- Can view as “**is-a**” relationship
- Represented by line ending in (open) triangle



**Laptop, Desktop, PDA inherit
state & behavior from Computers**

Exercise



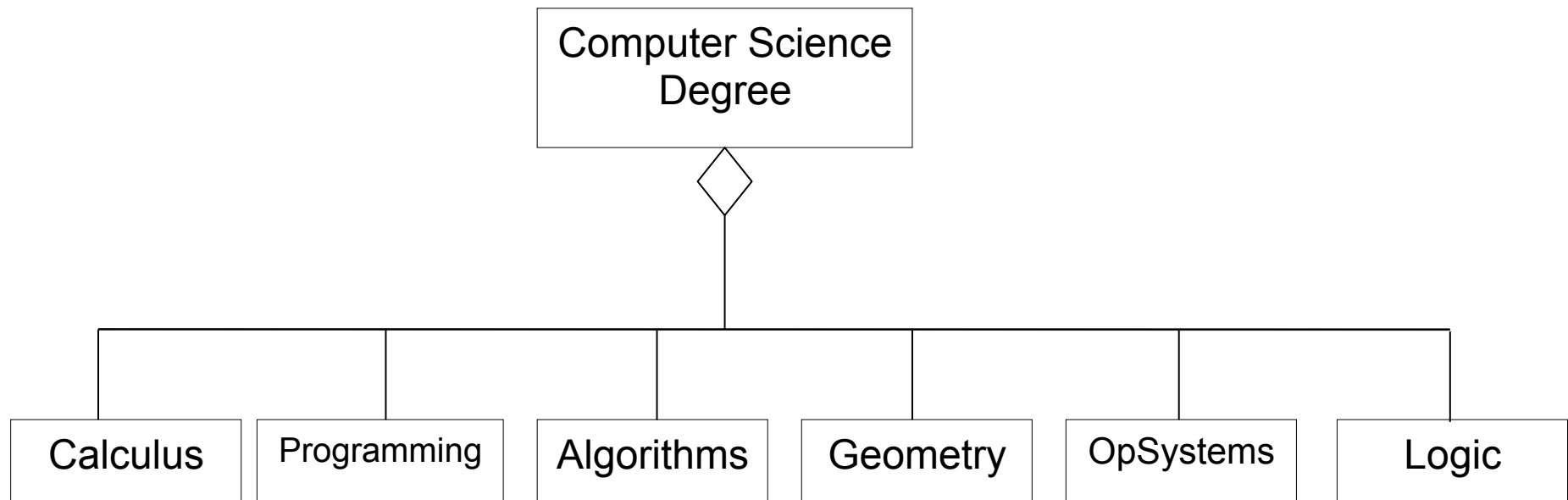
- Draw a class diagram showing inheritance relationships among classes Person, Employee, and Manager
- Draw a class diagram showing inheritance relationships among classes Person, Student, Professor, and Assistant

Aggregation

- An *aggregation* is a special form of association representing *has-a* or *part-whole* relationship
- It distinguishes the whole (aggregate class) from its parts (component class)
- WARNING: an aggregation does not bind the parts' lifetime to the whole (they can exist separately)

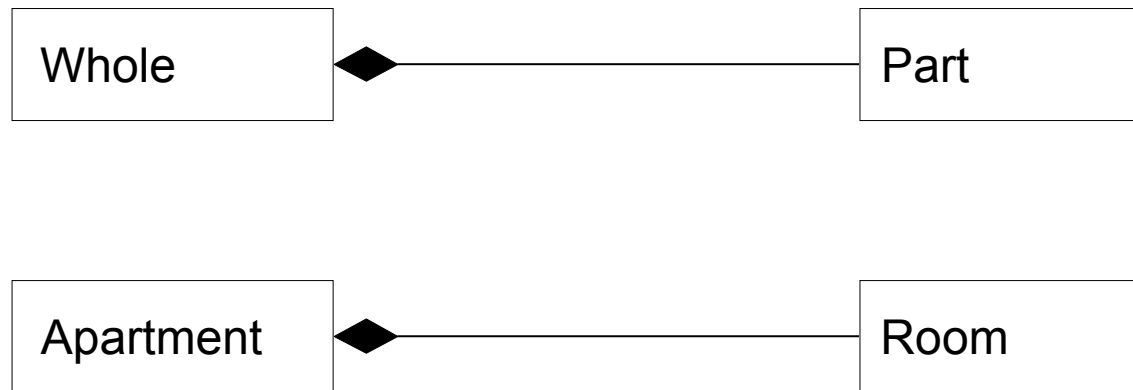


Example of an aggregation hierarchy

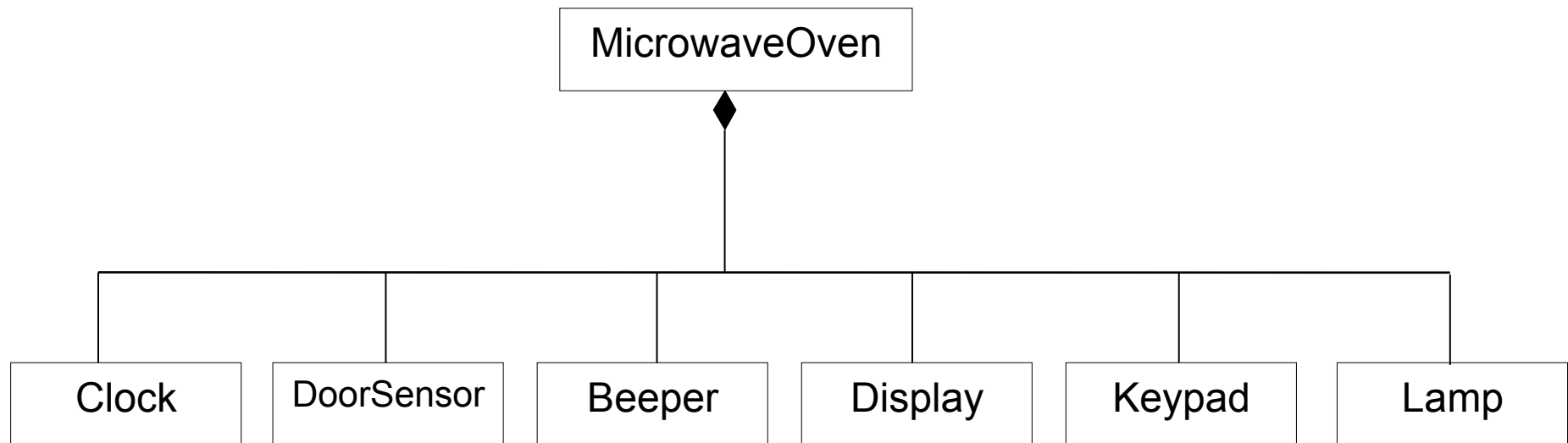


Composition

- A *composition* is a stronger form of aggregation
- It implies exclusive ownership of the component class by the aggregate class
- The lifetime of the parts is entirely included in the lifetime of the whole (a part can not exist without its whole)



Example of a composition hierarchy



Exercise



Imagine some aggregation or composition relationships among the following classes and draw a corresponding class diagram

- Employee
- Manager
- Office
- Department

Use Case Diagrams

Example

- A chess program could be “stand-alone”, “client-server”, “agent based”, etc.
- Its behavior should always be coherent with the rules of chess
- What is its **goal**? To play and win a chess game against an opponent

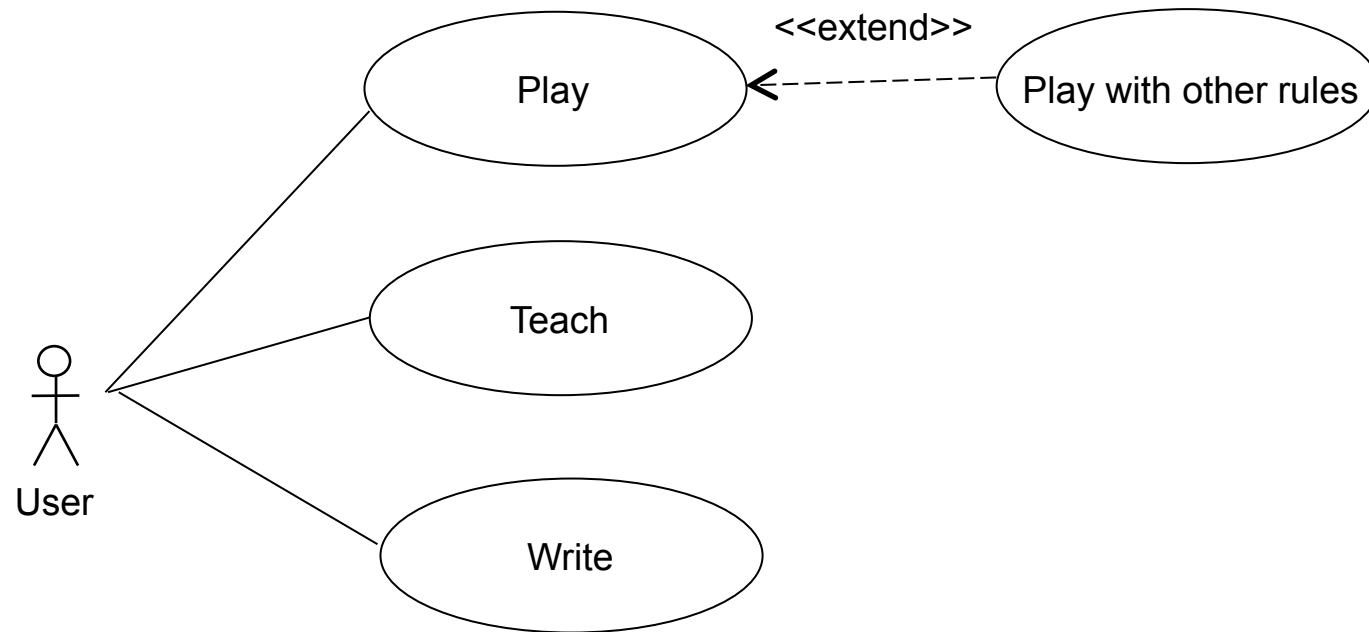
Goals and Responsibilities

The very same chess program, with identical structure and behavior, could be used with a different GOAL?

- For instance, could it be used to learn to play chess?
 - **Responsibility of the program**: teach chess
- Or to write a chess book, like a chess game editor?
 - **Responsibility of the program**: write chess texts
- Or to play a game of loser's chess (where who is checkmated wins)?
 - **Responsibility**: play games with rules slightly different from chess

Each responsibility corresponds to (at least) a use case

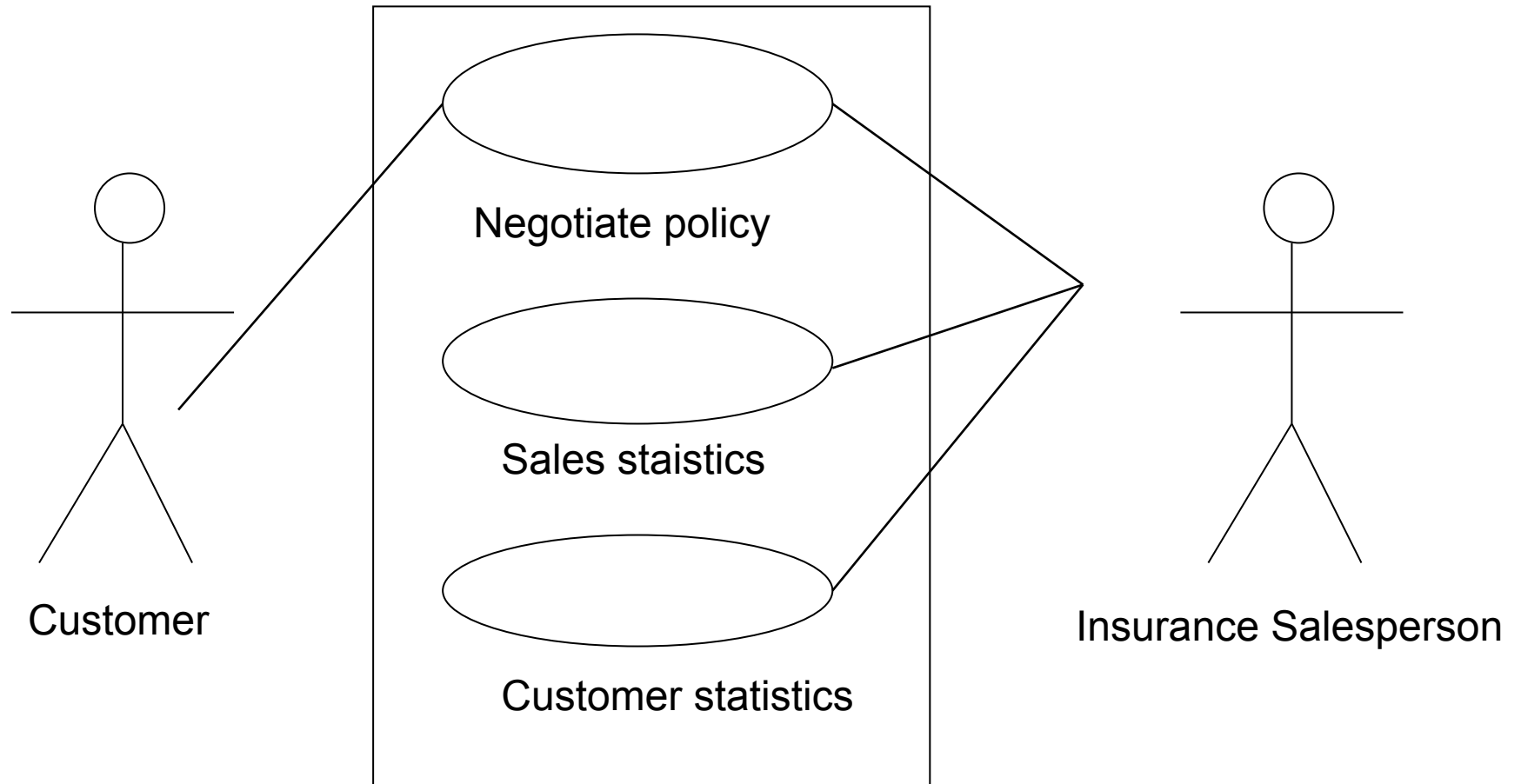
From responsibilities to use cases



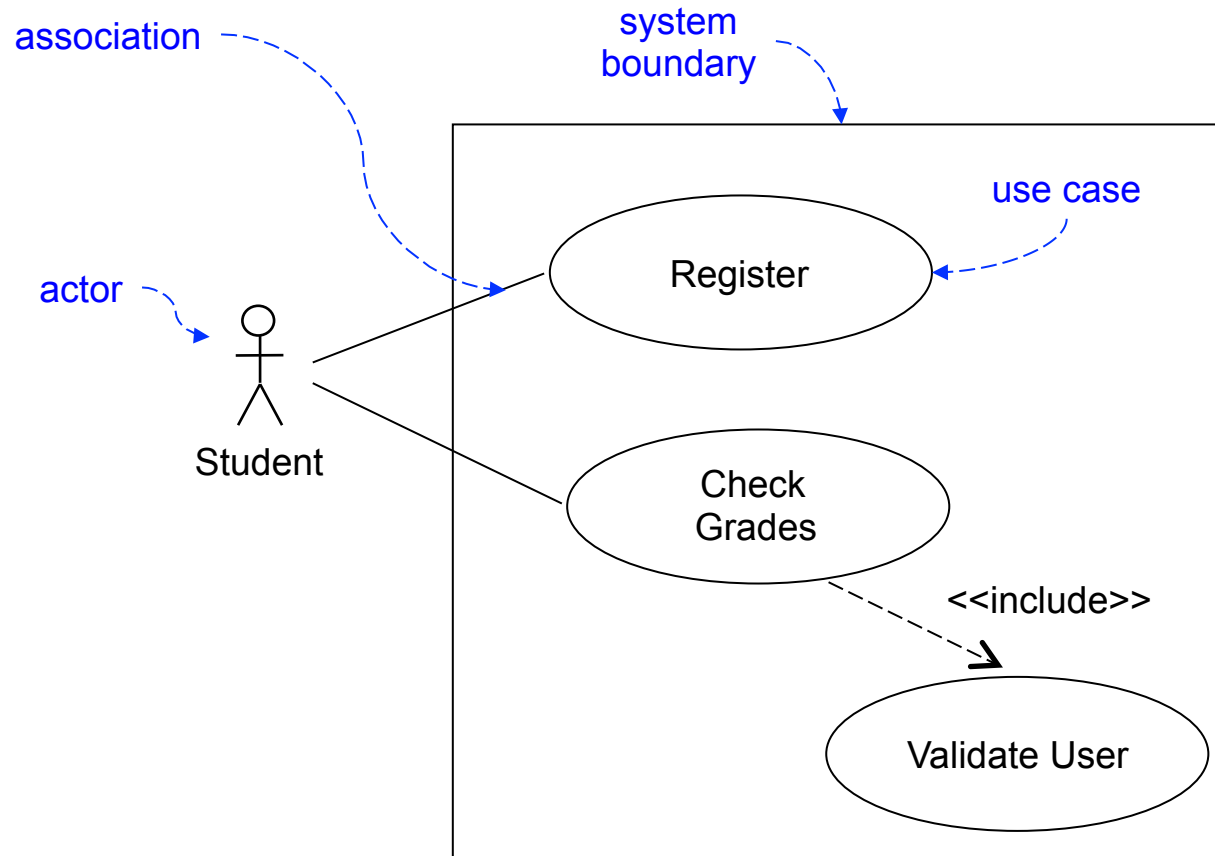
Use Case diagram

- It describes the externally observable behavior of a system, as related to **requirements**
- It describes the main interactions between the system and external entities, including users and other systems
- It is a summary of the main scenarios where the system will be used
- It describes the main user roles

Example



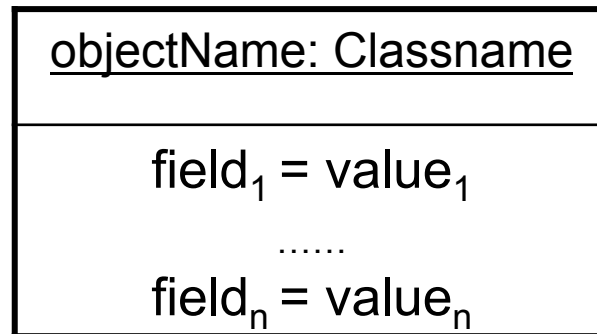
Use Case: elements



Behavioral Models

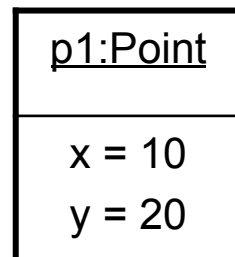
Notation for Objects

- Object: Rectangular box with one or two compartments
- Object name: object name/role name:class name

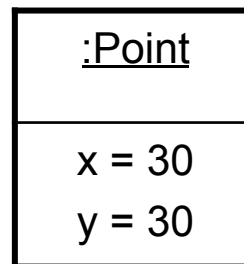


The top compartment shows the name of the object and its class.

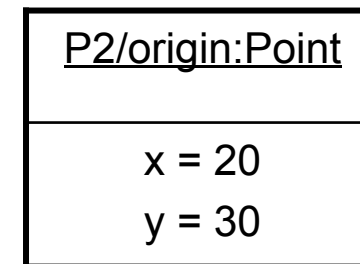
The bottom compartment contains a list of the fields and their values.



named



anonymous



**named
with role**

Object = Identity + State + Behavior

- Identity
 - Distinguishes an object from all other objects
- State
 - Consists of a set of *attributes* (or *fields*), which have names, types, and values
- Behavior
 - Defined by the set of *operations* (or *methods*) that may operate on the object
 - Each method has a name, a type, and a value, where
 - The **type** consists of the return type and the list of parameter types of the method, often called *signature*.
 - The **value** is the implementation of the method often expressed as a sequence of statements, in languages like Java and C++

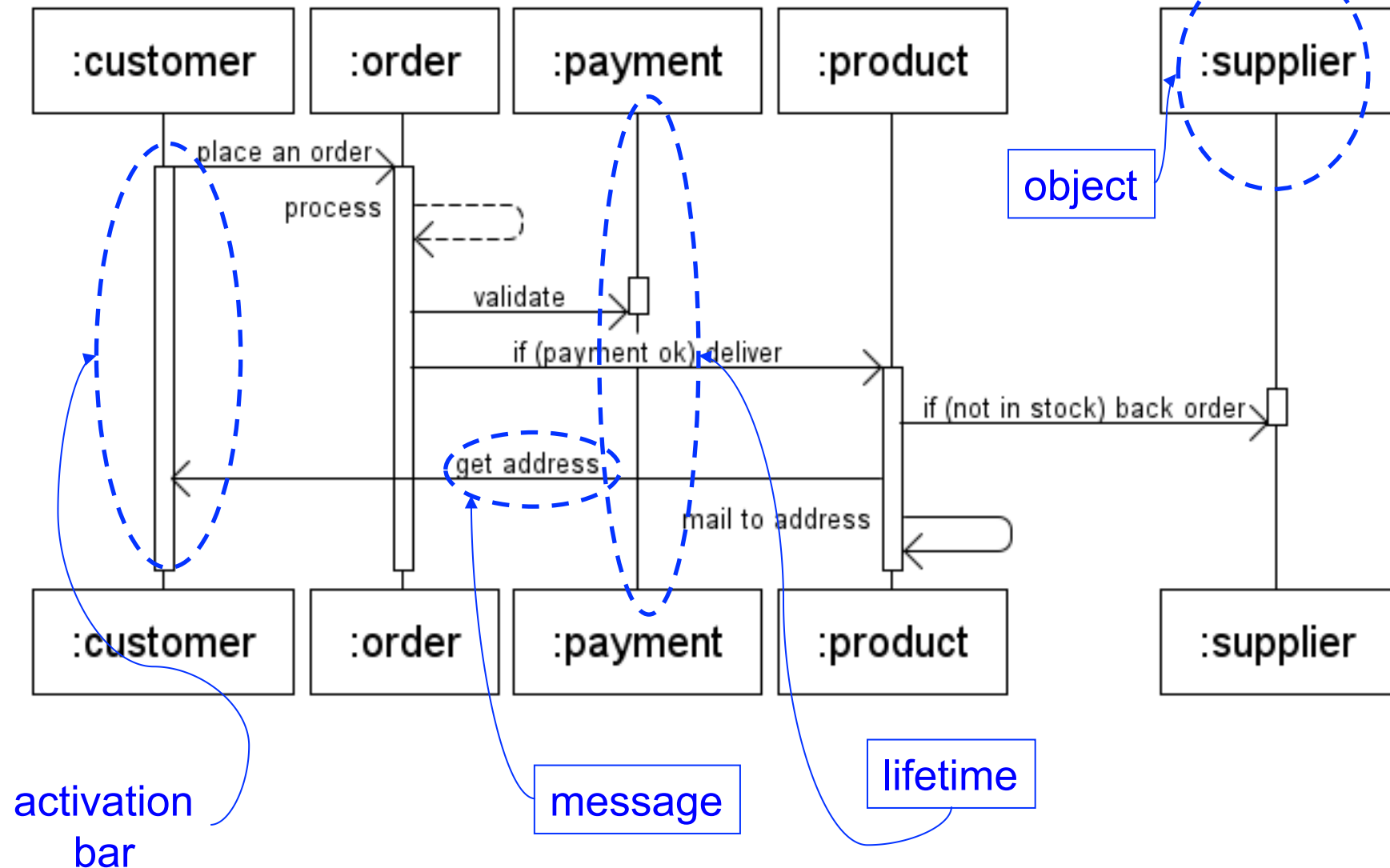
Interaction diagrams

- A use case diagram presents an **outside view** of the system
- The **inside view** of a system is shown by **interaction diagrams**
- Interaction diagrams describe how a use case is realized in terms of interacting objects
- Two types of interaction diagrams
 - **Sequence** diagrams
 - **Collaboration** (Communication) diagrams

Sequence diagram: main entities

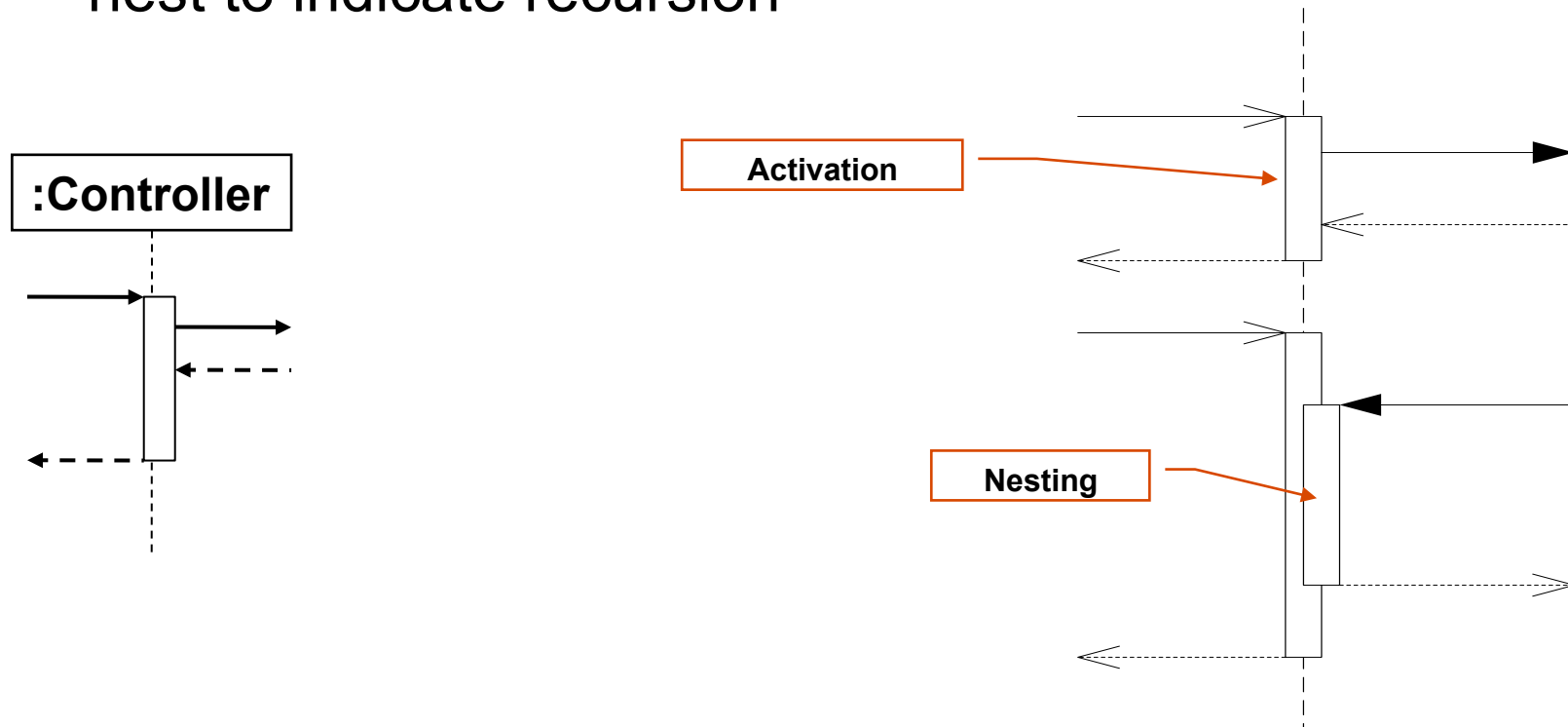
- **participant**: an object that acts in the sequence diagram
- **message**: communication between participant objects
- the **axes** in a sequence diagram:
 - horizontal: which object/participant is acting
 - vertical: time (down -> forward in time)

Sequence diagram: elements



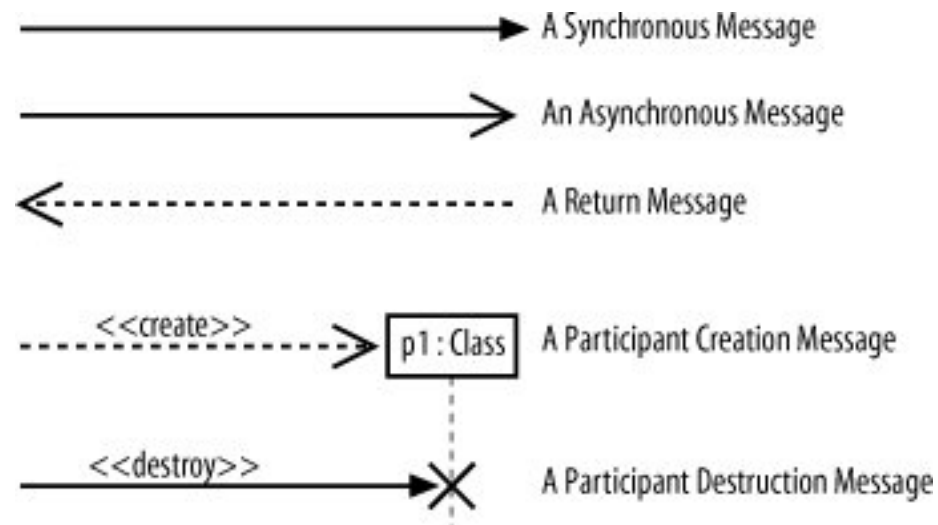
Indicating method calls

- **activation**: thick box over object's life line
 - The object is running its code, or it is waiting for another object's method to finish
 - nest to indicate recursion



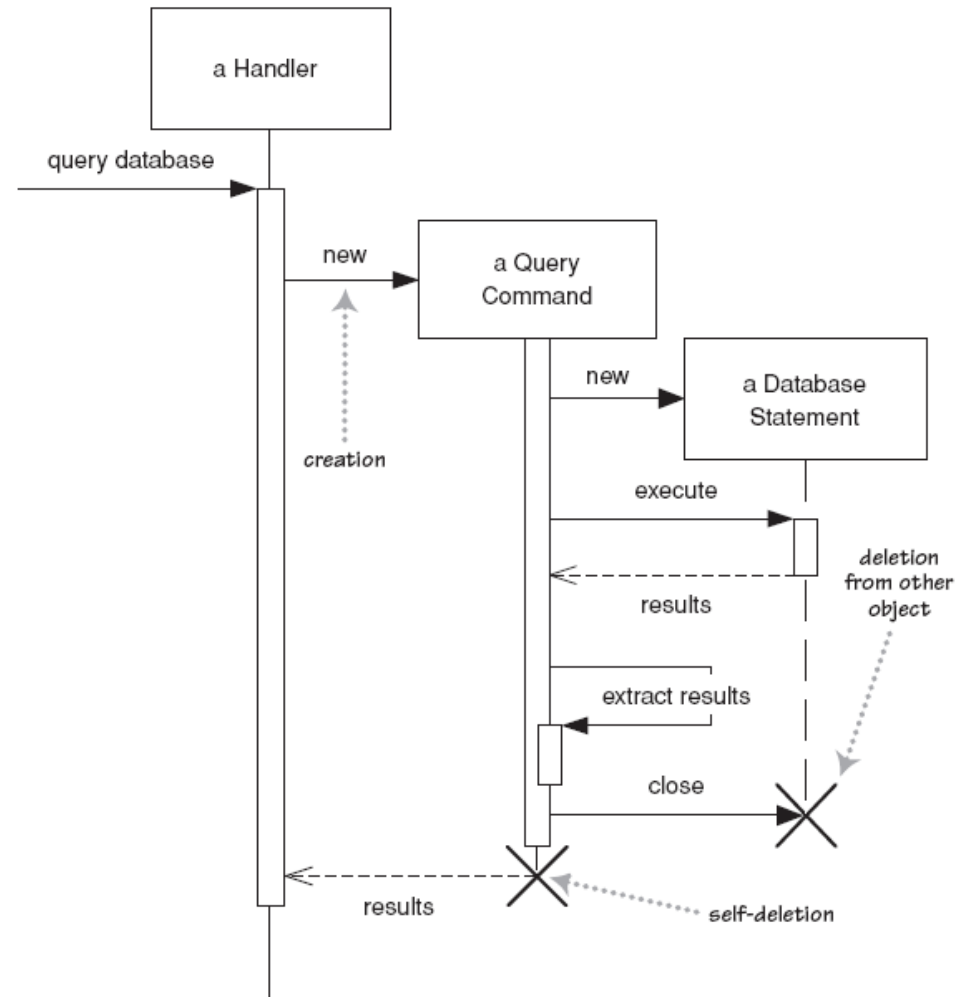
Arrows in a sequence diagram

- A sequence diagram describes sequences of method calls among objects
- There are several types of method calls

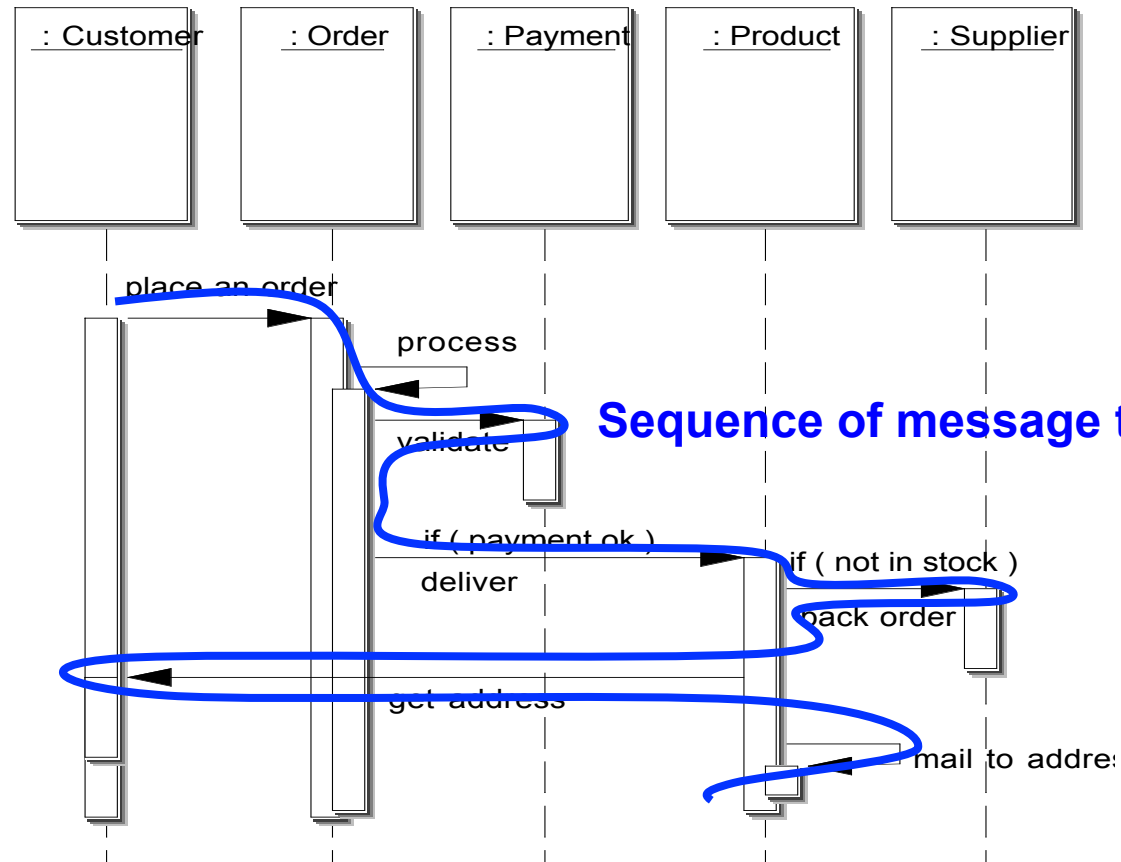


Lifetime of objects

- creation: arrow with 'new' label
 - an object created after the start of the scenario appears lower than the others
- deletion: an X at bottom of object's lifeline
 - Java does not explicitly delete objects; they fall out of scope and are garbage-collected



Sequence diagram: flow



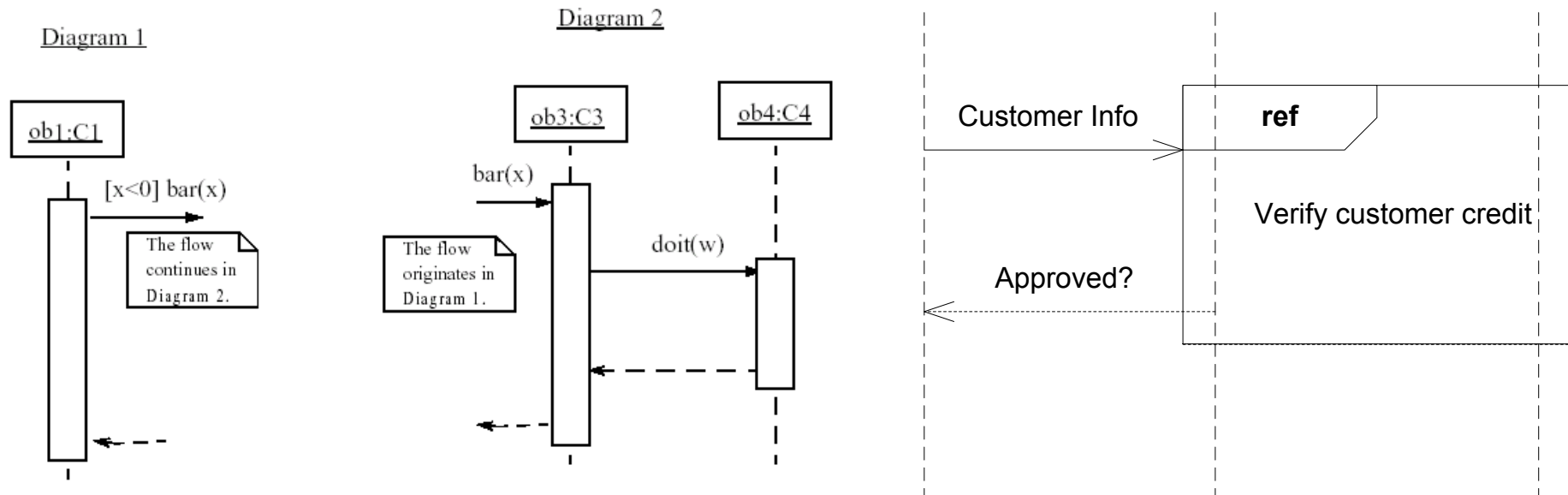
Sequence of message transmissions

Two Important SD elements

- Interaction Occurrence
 - Reusable interactions
 - Using ref element in the sequence diagram
 - To accommodate long sequence diagrams by invoking a SD from another SD
- Combined Fragments – for selection and loops in SD
 - Alt (equivalent of if)
 - Opt (equivalent of if-else)
 - Loop (equivalent of loop)

Linking sequence diagrams

- if a sequence diagram is too large or refers to another diagram, indicate it with either:
 - an unfinished arrow and a comment
 - a “**ref**” frame that names the other diagram

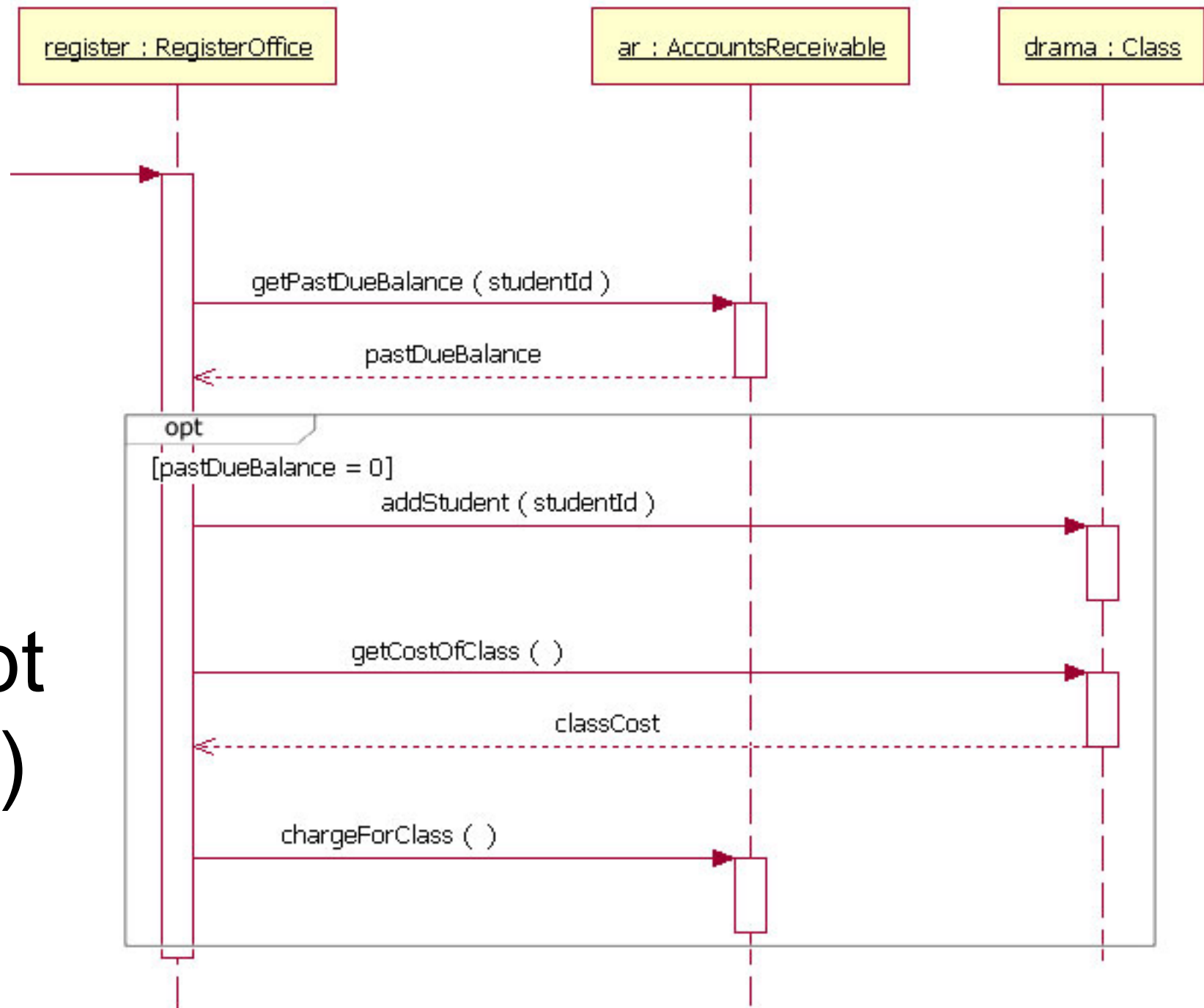


Conditionals and loops

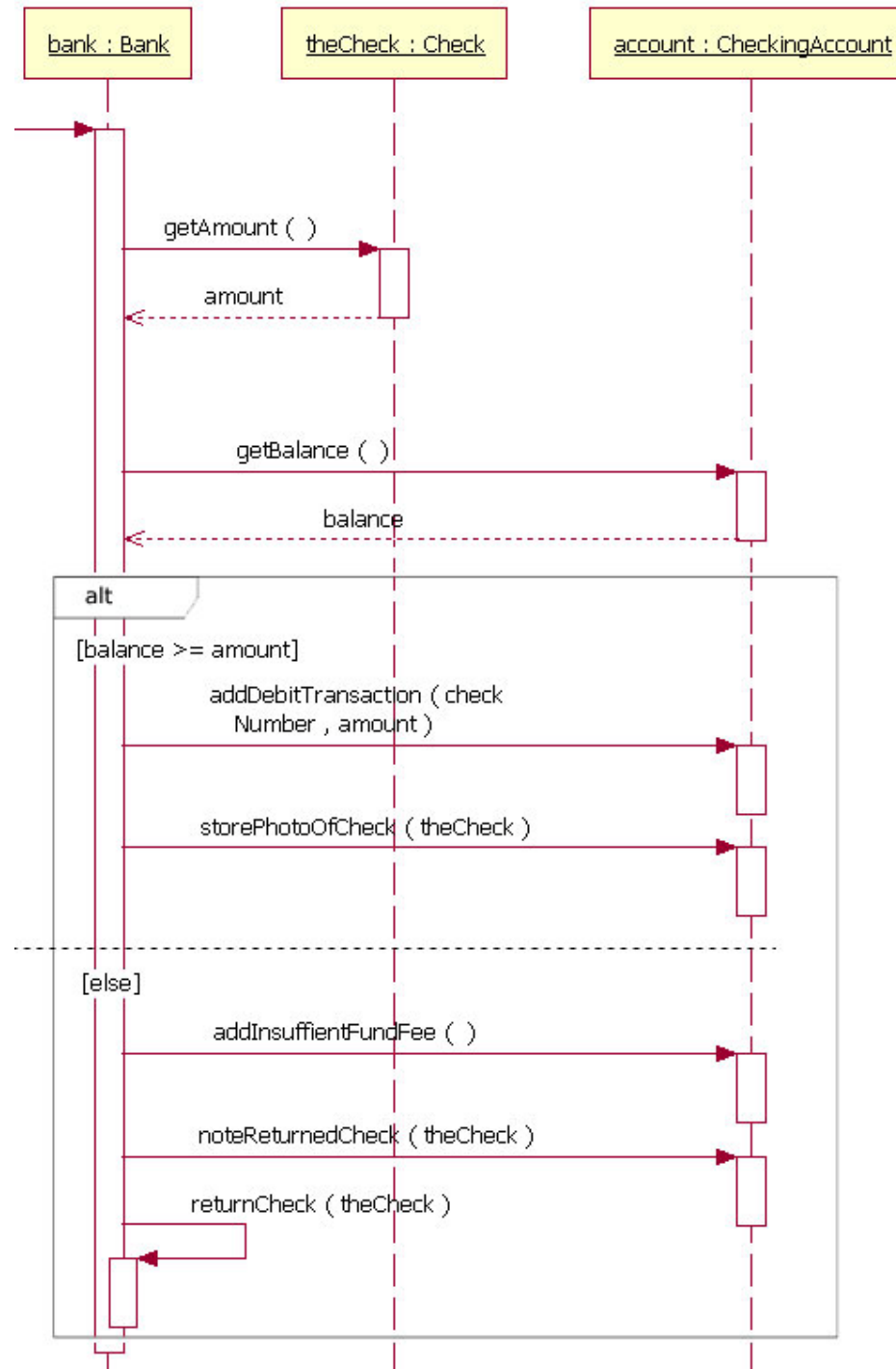
- frame: box around part of a sequence diagram to indicate selection or loop
 - if : (**opt**) [condition]
 - if-else: (**alt**) [condition] else [condition]
 - loop: (**loop**) [condition to loop over]

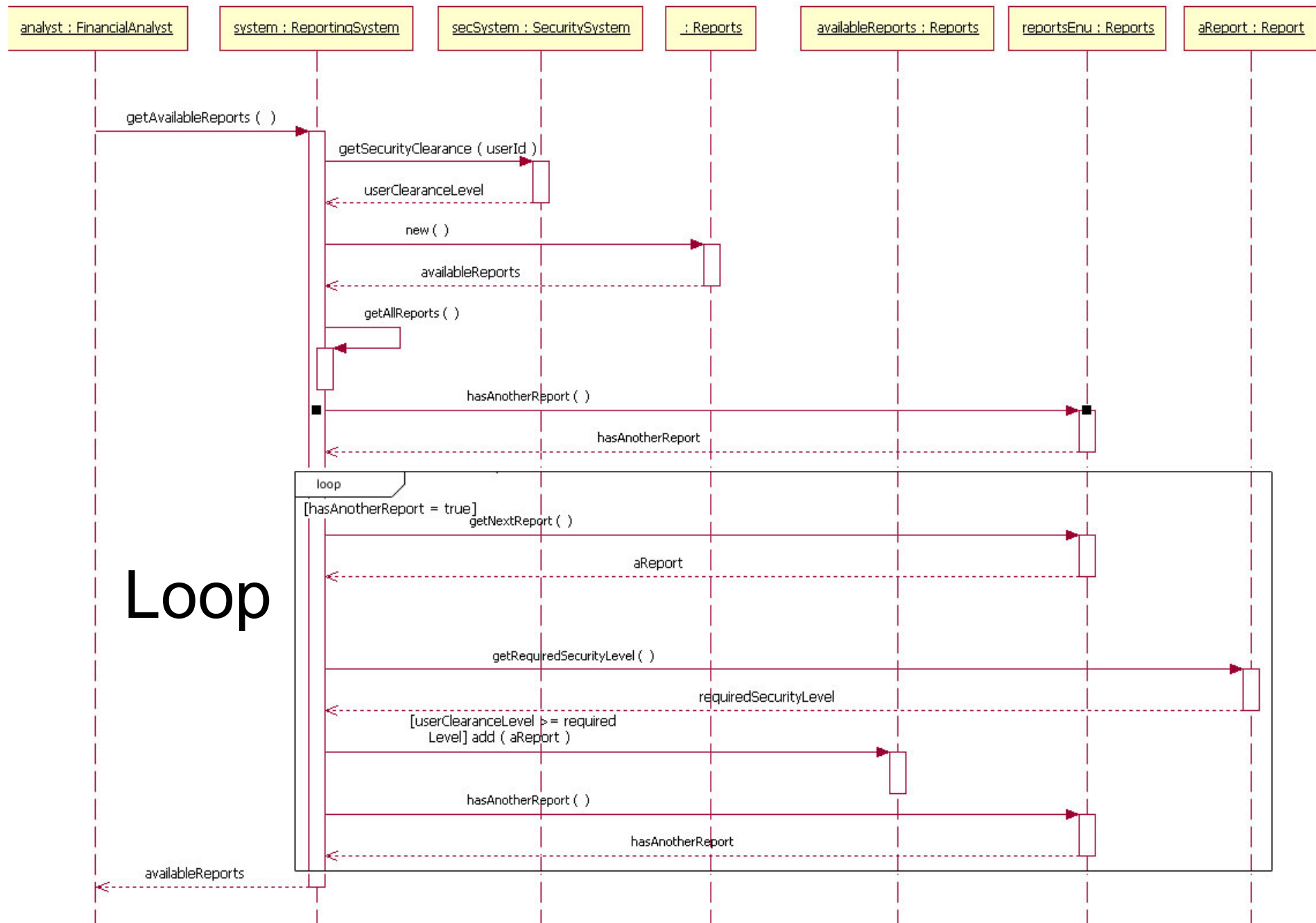
Examples: see the next three slides

Opt
(if)



Alt (if-else)





Activity Diagrams

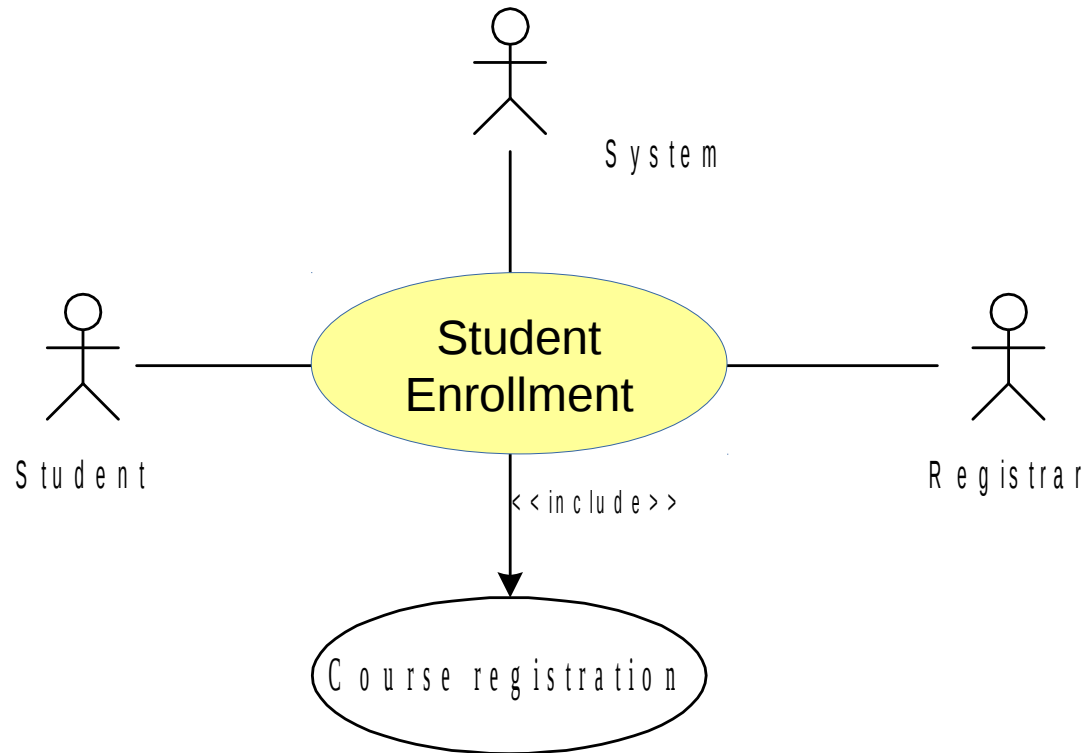
Activity Diagrams

- What is an activity diagram?
- Example: Student Enrollment System
- Activity diagram for a use case in SES
- Basic components in an activity diagram and their notations
- Managing the large activity diagram: Swim Lane
- Activity diagram vs. Flow chart

What is an Activity Diagram?

- Activity diagrams represent the dynamic (behavioral) view of a system. An Activity diagram is essentially a flowchart showing the flow from activity to activity.
- Activity diagrams can model sequential / concurrent behaviors in the system.
- Activity diagrams are typically used for
 - business (transaction) process modeling,
 - modeling the logic captured by a single use-case or usage scenario.
 - representing flows across use cases or to represent flow within a use case.
 - Work-flows and Data-flows, and Complex algorithms.
- UML activity diagrams are considered to be the OO-equivalent of flow chart and data flow diagrams in function-oriented design approach
- Activity diagram contains activities, transitions between activities, decision points, synchronization bars, swim lanes and much more...

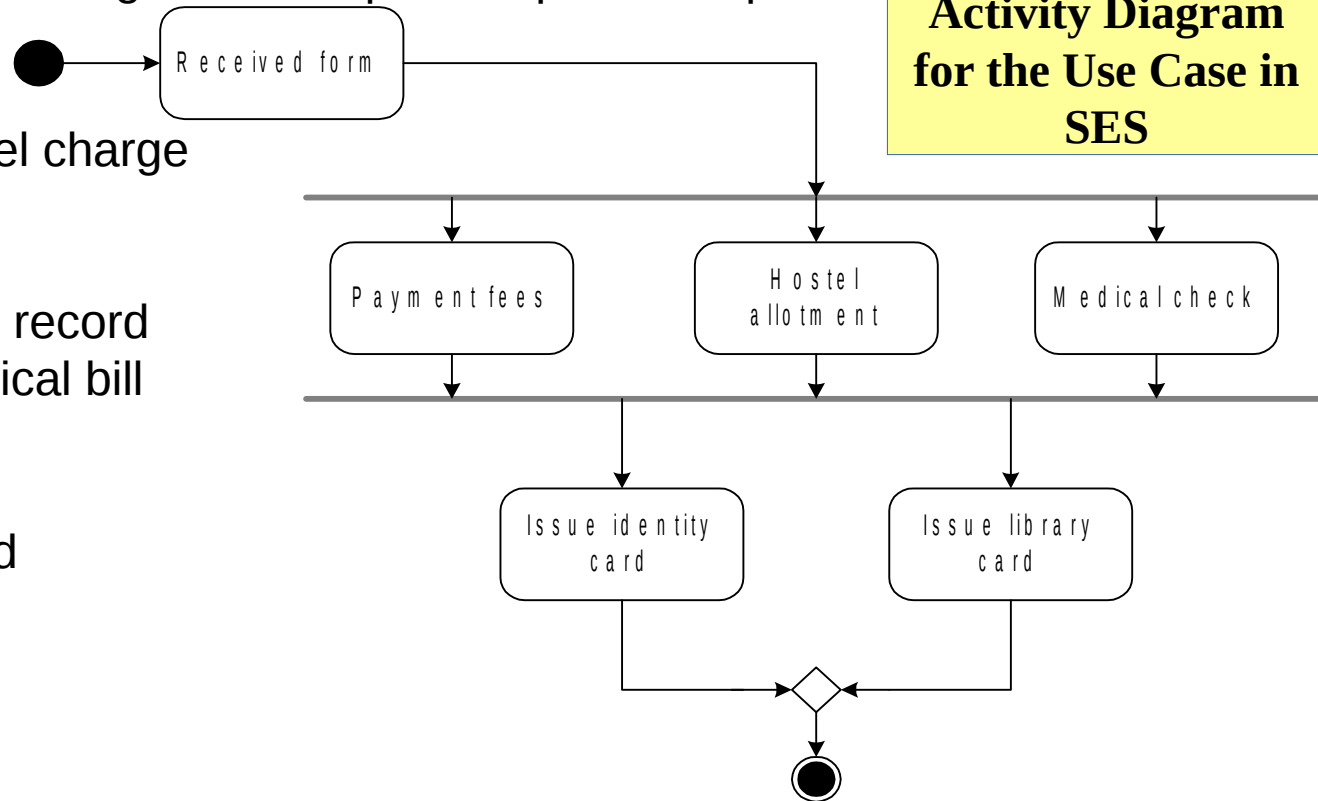
Student Enrollment System (SES)



Recall the concepts of DFD as a hierarchical model – Activity diagrams follow it.

Student Enrollment System (SES)

- Here different activities are:
 - Received enrollment form filled by the student
 - Registrar checks the form
 - Input data to the system
 - System authenticate the environment
 - Pay fees by the student
 - Registrar checks the amount to be remitted and prepare a bill
 - System acknowledge fee receipts and print receipt
 - Hostel allotment
 - Allot hostel
 - Receive hostel charge
 - Allot room
 - Medical check up
 - Create hostel record
 - Conduct medical bill
 - Enter record
 - Issue library card
 - Issue identity card



Basic Components in an Activity Diagram



- **Initial node**

- The filled circle is the starting point of the diagram. An activity may have more than one initial node.
- An initial node is a control node at which flow starts when the activity is invoked.



- **Final node**

- The filled circle with a boarder is the ending point. An activity diagram can have zero or more activity final state; the first one reached stops all flows in the activity.

- **Activity and Action (Different ?)**

Action

Activity

- The rounded rectangle/ oval represents activities that occur. An activity is not necessarily a program, it may be a manual process/step also.
- It does something. There is an automatic transition on its completion. A transition (arrows between actions) supports modelling of control flow.

- **Flow/ edge**

- The arrows in the diagram. No label is necessary.

Understanding Actions and Activity

- Action state

- It represents a single step within an activity that is not further decomposed within the activity, i.e., It can't be decomposed.
- It is not interrupted on any event, are atomic and take very less time.
- It will not begin execution until all of its input conditions are satisfied.
- It is a special case of activity state that cannot be decomposed.

- Activity states

- Activity is non-atomic.
- It is a composite of other activity states and actions, i.e., it represents a behaviour that is composed of individual elements that are actions.

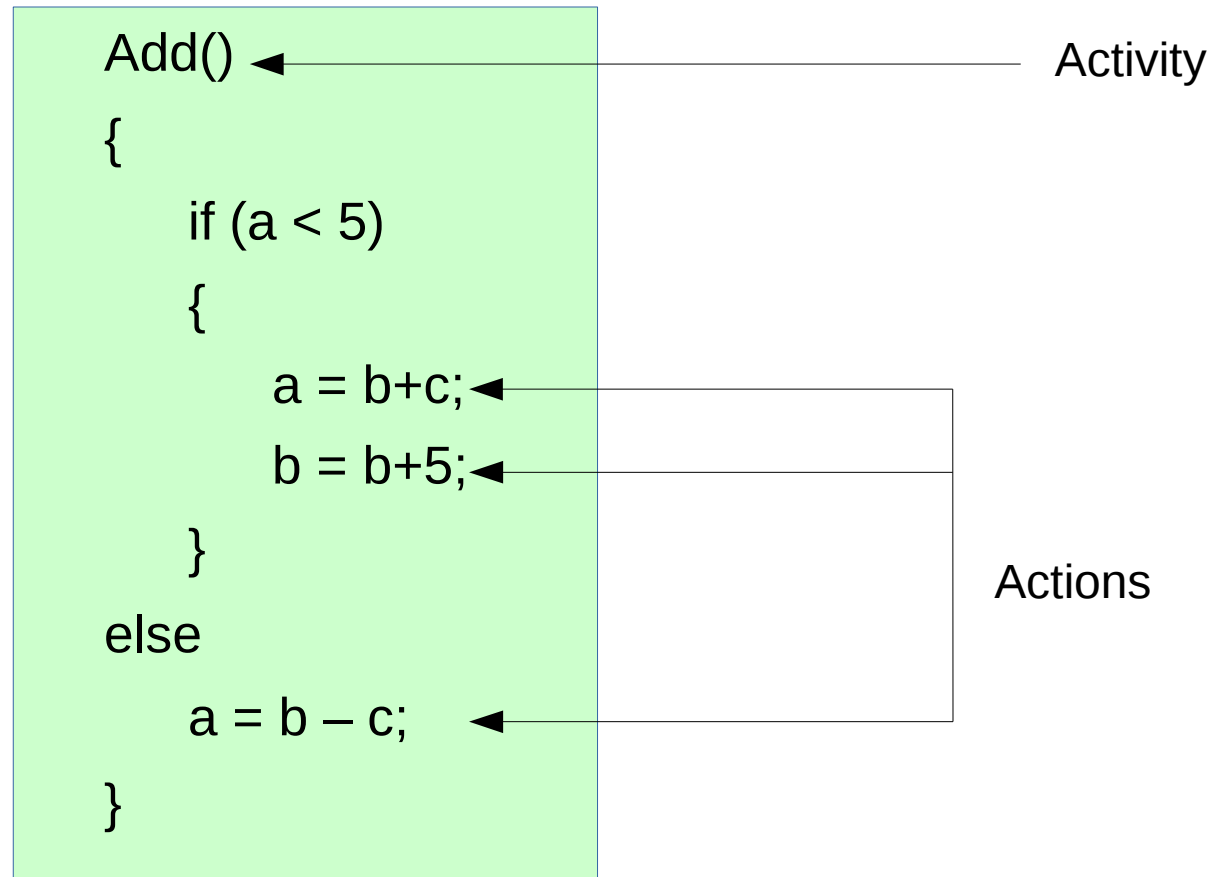
Bid plan

Action state

ProcessBill ()

Activity state

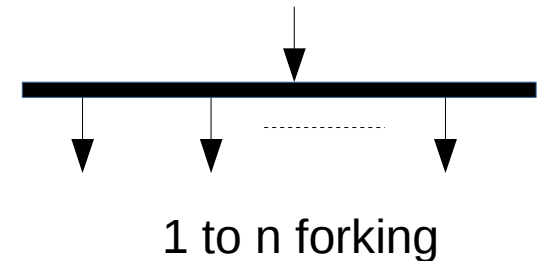
Analogy of Activity / Action with Function



Basic Components in an Activity Diagram

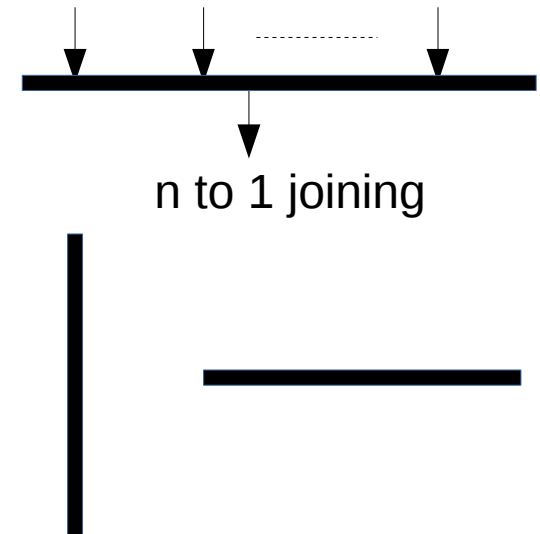
- **Fork**

- A black bar (horizontal/ vertical) with one flow going into it and several leaving it. This denotes the beginning of parallel activities.
- A fork node is a control node that splits a flow into multiple concurrent flows.



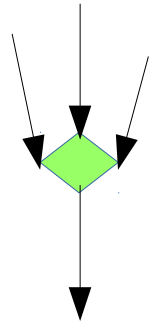
- **Join**

- A black bar with several flows entering it and one leaving it. This denotes end of parallel activities.
- A join node is a control node that synchronizes multiple flows.



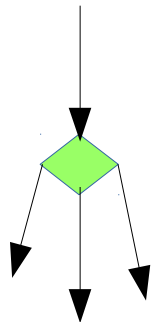
Basic Components in an Activity Diagram

- **Merge**



- A diamond with several flows entering and one leaving - It has multiple incoming edges and a single outgoing edge.
- The implication is that all incoming flow to reach this point until processing continues.
- It is a control node that brings together multiple alternate flows.
- It is not used to synchronize concurrent flows but to accept one among several alternate flows.

- **Decision** ([Recall Flowcharts](#))



- A diamond with one flow entering and several leaving - It accepts tokens on an incoming edge and presents them to multiple outgoing edges.
- The flow leaving includes conditions as yes/ no state or boolean expressions.
- Which of the edges is actually traversed depends on the evaluation of the guards on the outgoing edges.

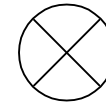
Basic Components in an Activity Diagram

- Difference between Join and Merge
 - A join is different from a merge in that the join synchronizes two inflows and produces a single outflow. The outflow from a join cannot execute until all inflows have been received.
 - A merge passes any control flows straight through it. If two or more inflows are received by a merge symbol, the action pointed to by its outflow is executed two or more times.

Basic Components in an Activity Diagram

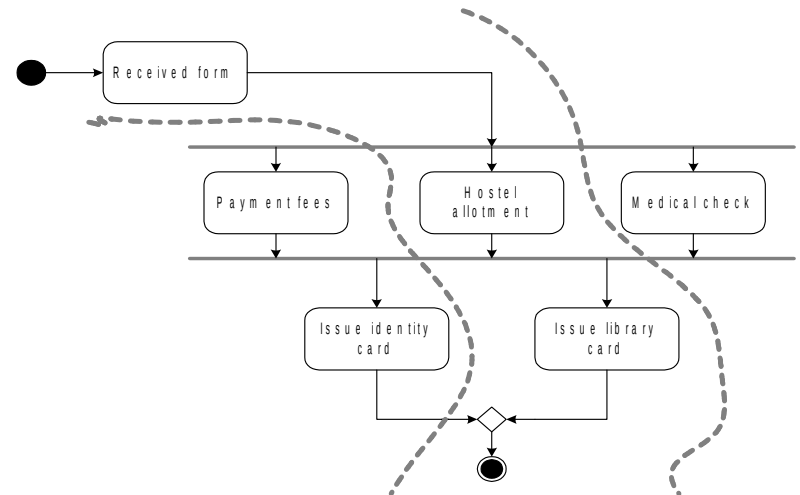
- **Flow final**

- The **circle with X** on it.
- This indicates that Process stops at this point.

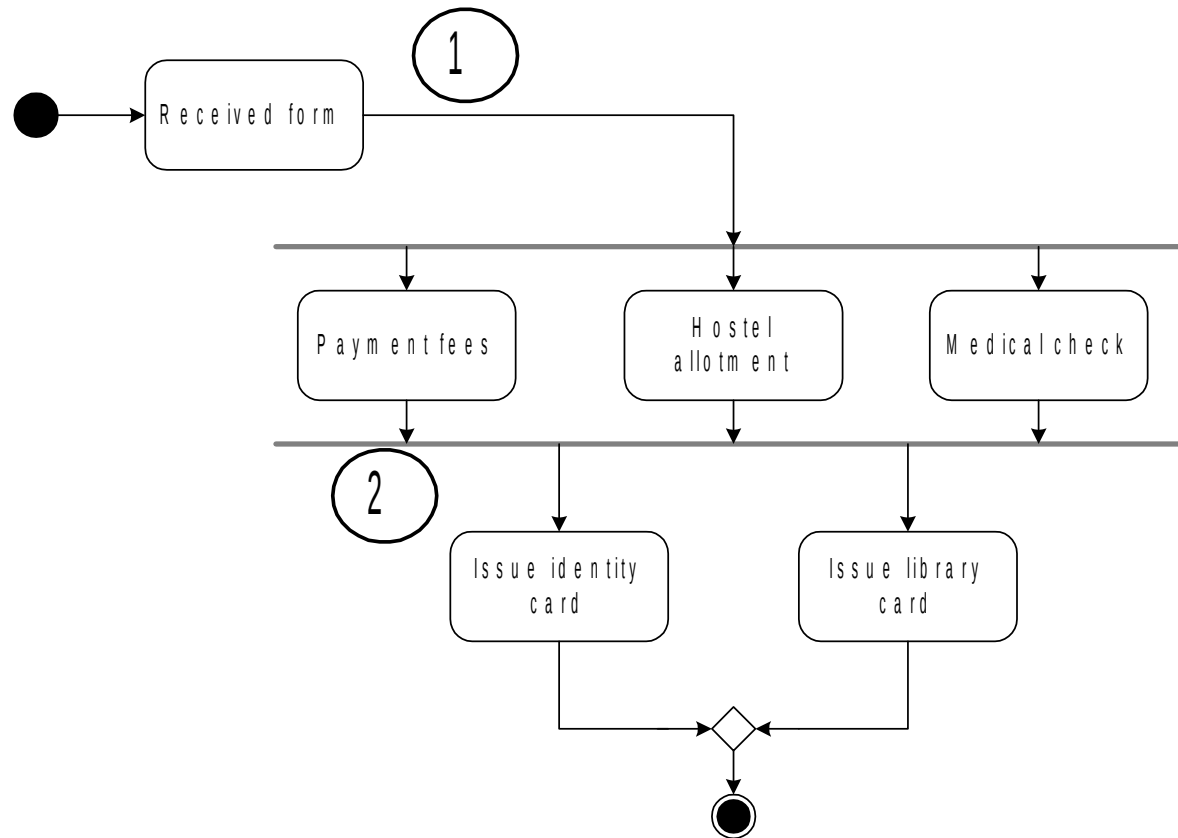


- **Swim lane**

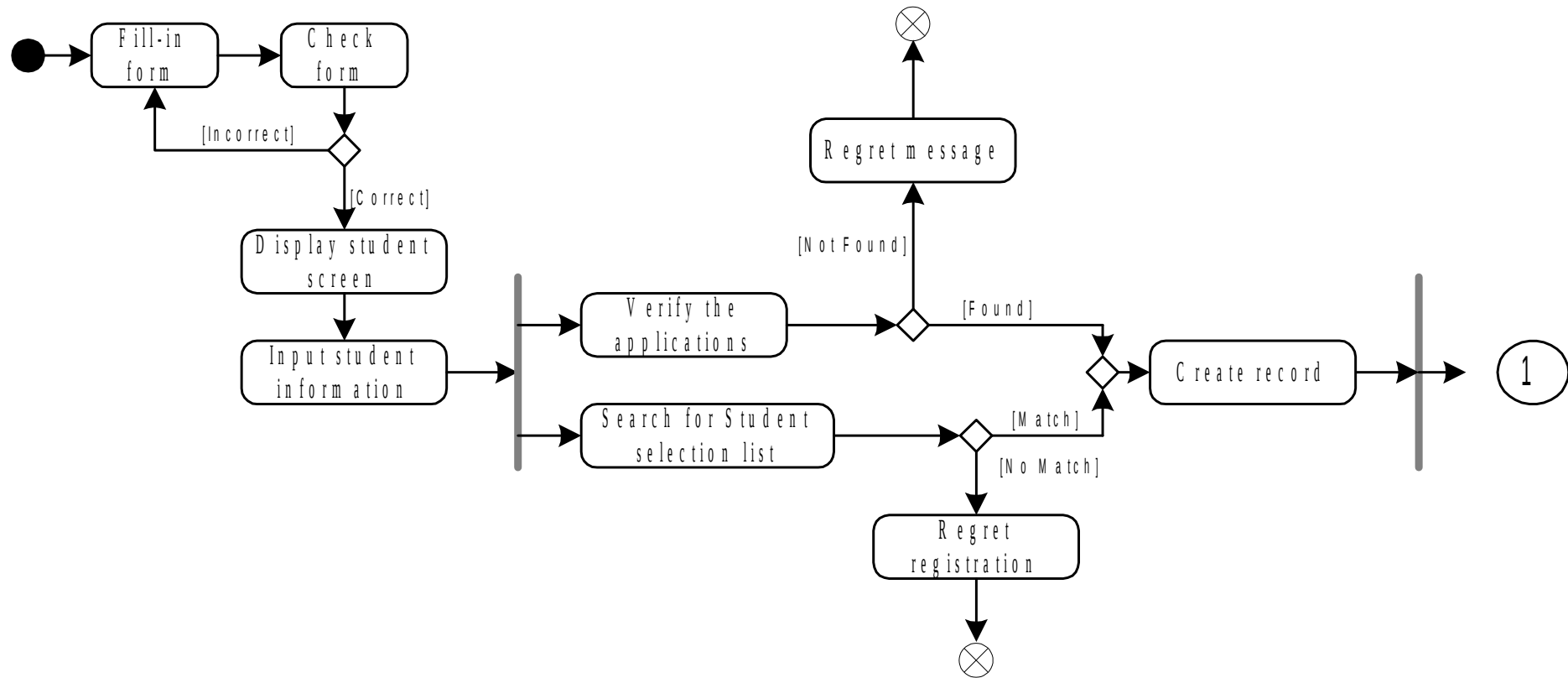
- A partition in activity diagram by means of dashed line, called swim lane.
- This swim lane may be horizontal or vertical.
- **Similar to swimming lanes.**



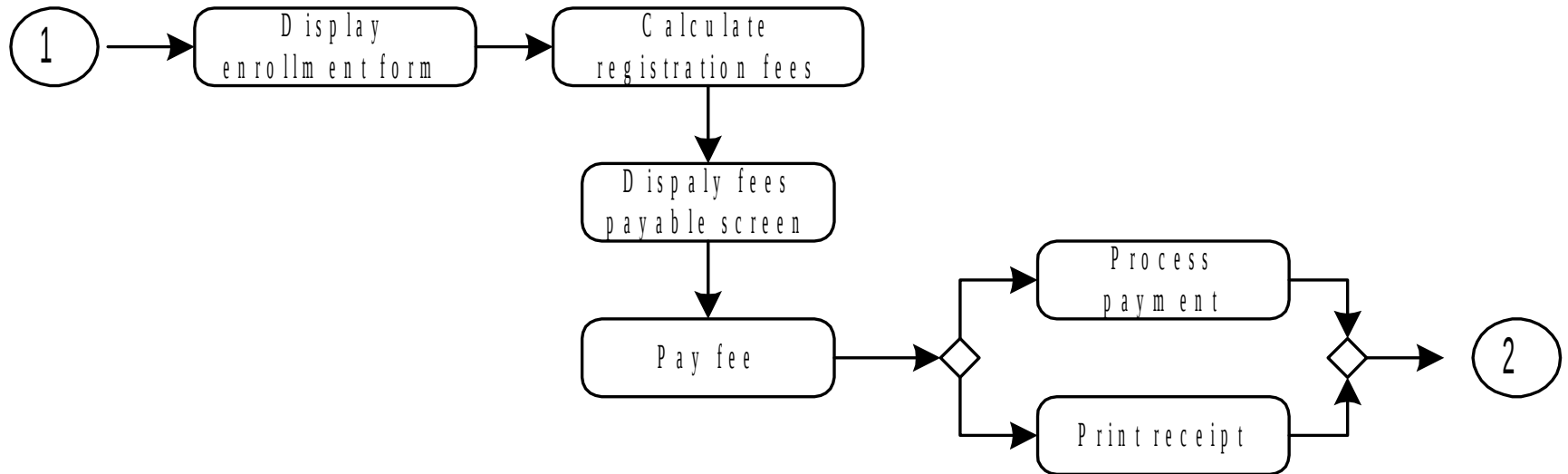
Detailed Activity Diagram of SES



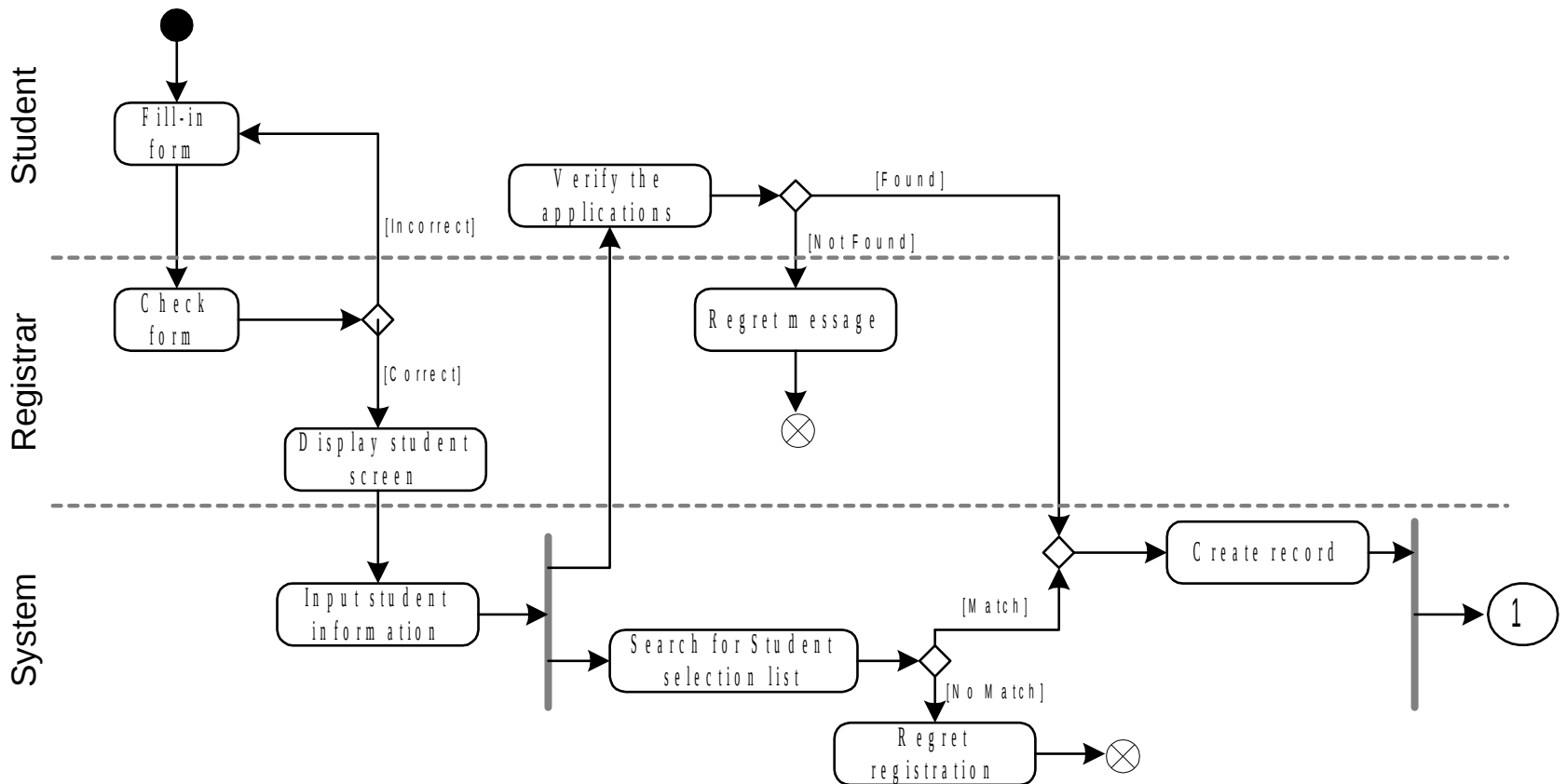
Detailed Activity Diagram of SES



Detailed Activity Diagram of SES

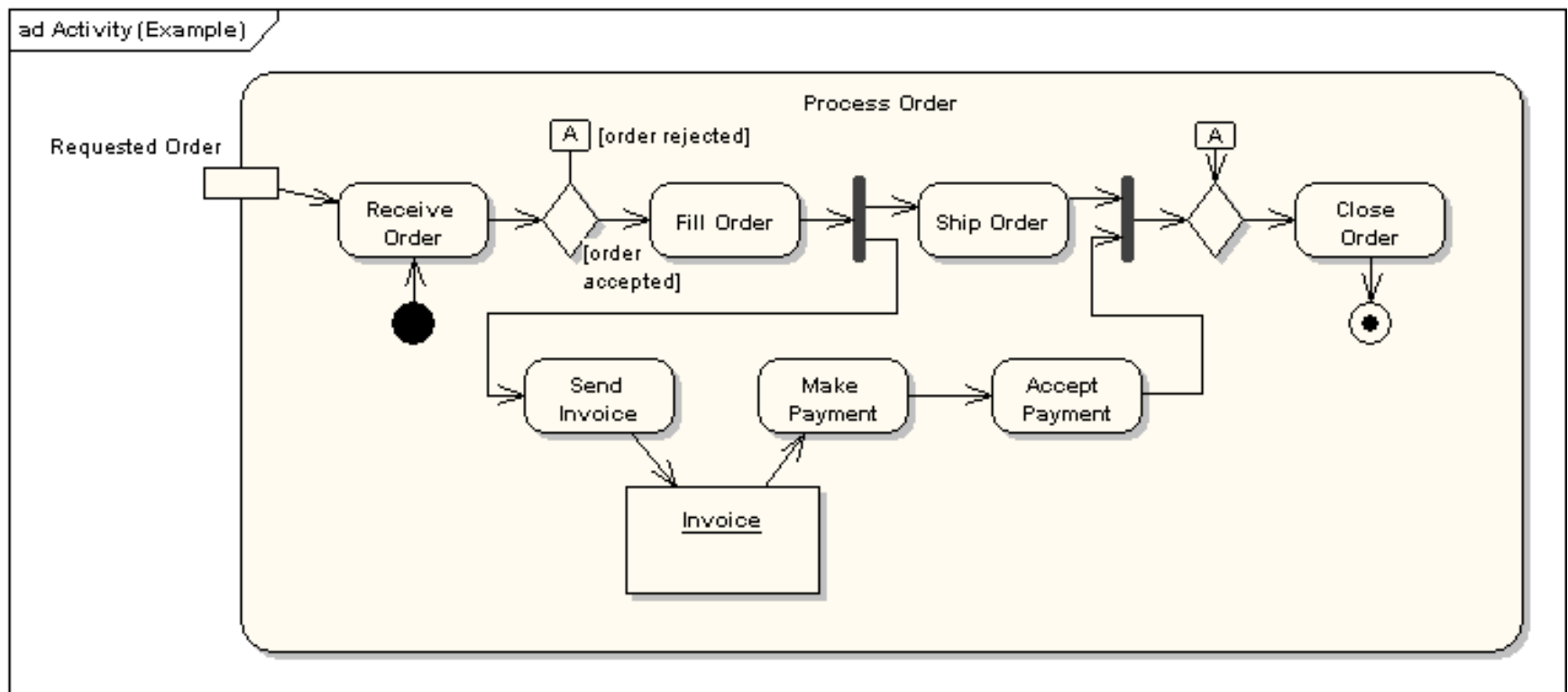


Activity Diagram of SES with Swim Lane



An Example

- Identify the business logic in the activity diagram shown below



Example Exercises

- Draw the activity diagrams for
 - Restaurant Management System
 - Library Information System
 - Bank ATM